



TM1415

Dimensioning and programming for ACOPOStrak

Requirements

Basic knowledge	Basic technical understanding
Training modules	TM210 - Working with Automation Studio TM415 - Introduction to mapp Axis
Software	Automation Studio 4.9 mapp Motion 5.13
Hardware	Automation Runtime Simulation

Table of contents

1 Introduction.....	5
1.1 Learning objectives.....	5
1.2 Safety guidelines.....	6
2 Creating a layout and dimensioning a network.....	7
2.1 Layout design.....	7
2.2 Network.....	8
3 Initial project with ACOPOStrak.....	11
3.1 Term definitions.....	11
3.2 Configuration of ACOPOStrak.....	12
3.3 Enabling ACOPOStrak in the simulation.....	15
3.4 HMI application with Scene Viewer.....	17
4 ACOPOStrak electrical dimensioning.....	19
4.1 Sizing in the simulation.....	19
4.2 Power supply structure.....	19
4.3 Configuration.....	19
4.4 Results.....	24
5 Access to user data with process points.....	25
5.1 Process points.....	25
5.2 User data.....	25
5.3 Integration into the application.....	26
6 Modular application using process stations.....	28
6.1 Concept of the process station.....	28
6.2 Programming a process station.....	28
7 Enhancing a layout.....	30
7.1 Adjustments in System Designer.....	30
7.2 Configuration of the additional segments.....	30
8 Types of movements.....	32
8.1 Elastic movements.....	32
8.2 Rigid movements.....	32
9 Process points as a barrier.....	34
9.1 Types of obstacles.....	34
9.2 Barriers.....	34
10 ACOPOStrak couplings.....	35
10.1 General information.....	35
10.2 Configuring a coupling object.....	35
10.3 Programming.....	35
10.4 Coupling between an axis and shuttle.....	37
10.5 Coupling between two shuttles.....	38
11 Interlinking.....	40
11.1 Concept.....	40
11.2 Implementation.....	40

11.3 Accuracy.....	40
12 Summary.....	42
13 Solutions and appendix.....	43
13.1 Solution: POWERLINK - Distribution.....	43
13.2 Solution: Electrical dimensioning.....	43
13.3 Solution: Accessing the user data.....	45
13.4 Basic step sequencer for process stations.....	48
13.5 Solution: Developing a process station.....	49
13.6 Solution: Extending the assembly with diverters.....	49
13.7 Solution: Process station with rigid movement.....	50
13.8 Solution: Coupling between an axis and shuttle.....	52
13.9 Solution: Coupling between two shuttles.....	55

1 Introduction

ACOPOStrak is an intelligent track system based on linear motor technology that revolutionizes the flexibility of machines and systems and thus enables economic symbiosis between batch size 1 and mass production. Unique product features such as absolute design freedom, high-speed diverting or the intelligent system software enable parallel process stations and scalable machines. Due to the modular design, existing machines can be easily upgraded and additional processing stations added if required.

This training module deals with dimensioning, programming and configuration of ACOPOStrak. Programming and also a portion of dimensioning take place in the simulation environment.

Programming focuses on creating applications in a structured manner with multiple program tasks and actions in order to improve clarity, reusability and consistency of the application.

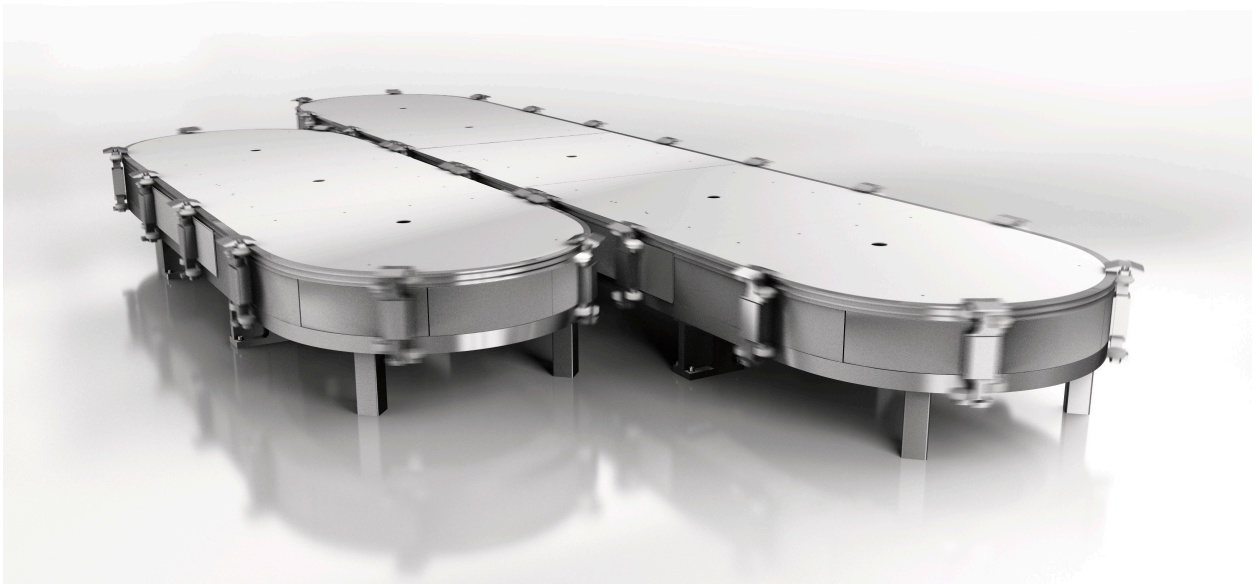


Figure 1: ACOPOStrak

The information and exercises provided here explain how ACOPOStrak should be programmed. An overview of the possibilities in the application should also be provided.

1.1 Learning objectives

This training module explains the basics for programming and dimensioning ACOPOStrak. There are numerous exercises available to help increase understanding. In addition, it will frequently refer to the extensive Automation Help, an invaluable reference for completing the exercises in this training module.

- Participants understand the software concept of ACOPOStrak.
- They know how the ACOPOStrak software components work.
- They are familiar with the simulation possibilities of ACOPOStrak.
- Participants can add and configure the ACOPOStrak hardware in the System Designer.
- Participants can configure the POWERLINK network correctly.
- Participants can use dynamic node allocation (DNA) with the ACOPOStrak hardware.
- Participants can add the ACOPOStrak configuration files and use them correctly.
- Participants are familiar with the available PLCopen function blocks and can use them in the project.
- They know how to add and remove shuttles at runtime and know the advantage of connecting ACOPOStrak to other systems.
- Participants are familiar with the different movement possibilities of shuttles and are able to configure them.
- Participants are familiar with the concept of process stations and are able to program them using the software.
- They are able to perform coupling between a shuttle and axis.
- They are able to perform coupling between two shuttles.
- Participants are familiar with and can apply the basics of ACOPOStrak dimensioning.
- Participants are able to carry out diagnostics tasks in the simulation using the Logger.

1.2 Safety guidelines

To ensure safe handling of ACOPOSTrak, the following instructions must be observed and hazardous areas secured and marked.

For additional safety guidelines, see the respective chapters in the user's manual.

Magnetic fields:





Electromagnetic fields can interfere with the proper functionality of electronic devices. This includes the operation of cardiac pacemakers, which could potentially lead to injury or death. Wearers of pacemakers are therefore prohibited from entering these areas.

Mechanical components:



Falling hardware components can cause personal injuries. For this reason, safety shoes must be worn when working with the mechanical system.

Shuttles in operation:



The incorrect behavior of track systems can trigger unintended and dangerous shuttle movements! Possible causes of this:

- Incorrect installation or faults when handling components
- Incorrect or incomplete wiring of the track system
- Defective components (segments, shuttles, position encoders, cables, etc.)
- Incorrect control (e.g. due to faulty software)



Shuttles can become detached from the guide system at high speed during the movement and cause substantial damage to property and personal injury! Possible causes of this:

- Poor weight distribution of the product / product carrier on the shuttle
- Adverse ratio of distances from centers of gravity to magnetic forces
- Poor geometry of the product / shuttle shelf
- Excessive weight of the transported product / product carrier
- Excessive speed and/or acceleration of the shuttle
- Product moving on the shuttle (sloshing, rolling, slipping)
- Nonobservance of limitations regarding the mounting orientation of the track
- Incorrect configuration/behavior of the track

Risk of burns:



When operating the shuttles on the track, the segments can heat up to temperatures hot enough to cause burns. For this reason, temperature-resistant safety gloves must be worn when replacing segments.

2 Creating a layout and dimensioning a network

2.1 Layout design

Options for creating an ACOPOStrak layout

The core of ACOPOStrak consists of 4 different segments; these allow for a high level of flexibility when creating a layout. This way, the system can be individually adapted to the needs of the customer.

The segments are labeled based on the segment transition in the counting direction. A straight transition is marked with the letter A. A curve transition is marked with the letter B.

The 4 segments:

- Straight segment (AA segment)
- Circular arc segment (BB segment)
- Curve entry segment (AB segment)
- Curve exit segment (BA segment)



Figure 2: The 4 segments of ACOPOStrak

The two curve segments allow for smooth transit around the curve, as the radius is slowly reduced from infinity (i.e. straight transition A) to the defined curve radius (i.e. curve transition B). The straight area takes up 105 mm of the segment length. This reduces both the mechanical and electrical load on the system.

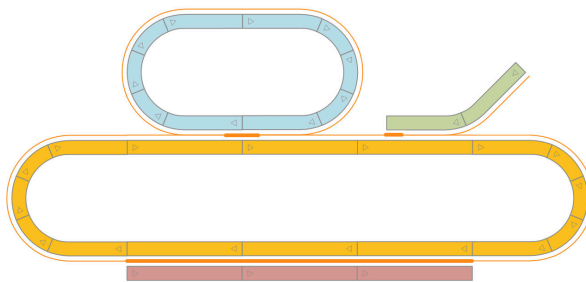


Figure 3: An assembly with 4 tracks

Tracks can then be created from the individual segments. These can be open or closed. One or more tracks are then joined together to form an assembly.

In the figure (Fig. 3 "An assembly with 4 tracks"), an assembly consisting of 4 tracks can be seen. Two tracks are closed as ovals and the other two are open tracks. The thick orange lines indicate the area where the shuttles can change tracks.

Notes regarding the layout creation

In order to guarantee a perfect movement of the shuttles, the following points must be observed when creating the layout:

- Switching from one track to another is only possible on straight segments and on the straight portion of curve segments.
- The parallel track in a diverter must be at least 90 mm long.
- Due to the coil arrangement, the offset of segment transitions should always be a multiple of 15 mm ($n * 15 \text{ mm}$). If this is not the case, the shuttle may change back and forth spontaneously (see Fig. 4 "Positioning a diverter").
- On a longer parallel route, the segment transitions should be directly opposite each other; otherwise, the shuttle will unintentionally change tracks due to the lower gravitational pull.
- If there is a segment transition in the diverter area, it must be at least 150 mm away from the segment end of the curve segment so that the track does not change spontaneously (see Fig. 4 "Positioning a diverter").

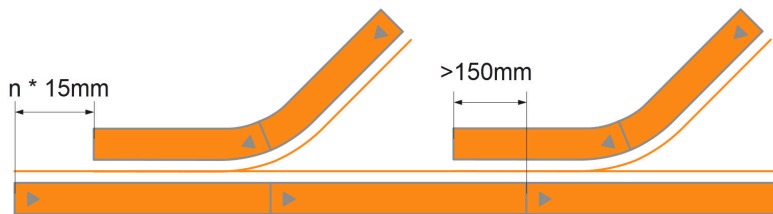


Figure 4: Positioning a diverter



After finding a suitable layout, an exact sketch of the layout should be created. All segment names, process areas and POWERLINK connections should be included in this sketch. Having a constant overview of the system will make the configuration and programming process easier at present and also the commissioning and diagnostics later on. Any changes should always be transferred to the sketch so that the overview is not lost.

Exercise: Creating the layout

In this exercise, an oval is to be extended so that the orientation of a shuttle can only be changed by moving it along the track.

- 1) Outline an oval with two 180° curves and a straight line in between.
- 2) Design the reverse path from the existing control blocks (straight, 45°, 90°, 135° and 180° curve).

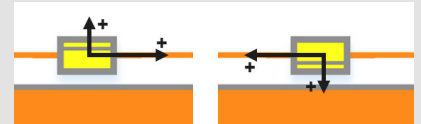


Figure 5: Changing the shuttle orientation

2.2 Network

POWERLINK - Distribution

Communication between the controller (usually an Automation PC) and the segments takes place via POWERLINK. POWERLINK cables are routed from the controller to the segments. From there, they are daisy-chained to the adjacent segments on the left and right. This reduces the number of cables in the control cabinet to a minimum. However, there are a few guidelines to be followed to ensure proper operation.

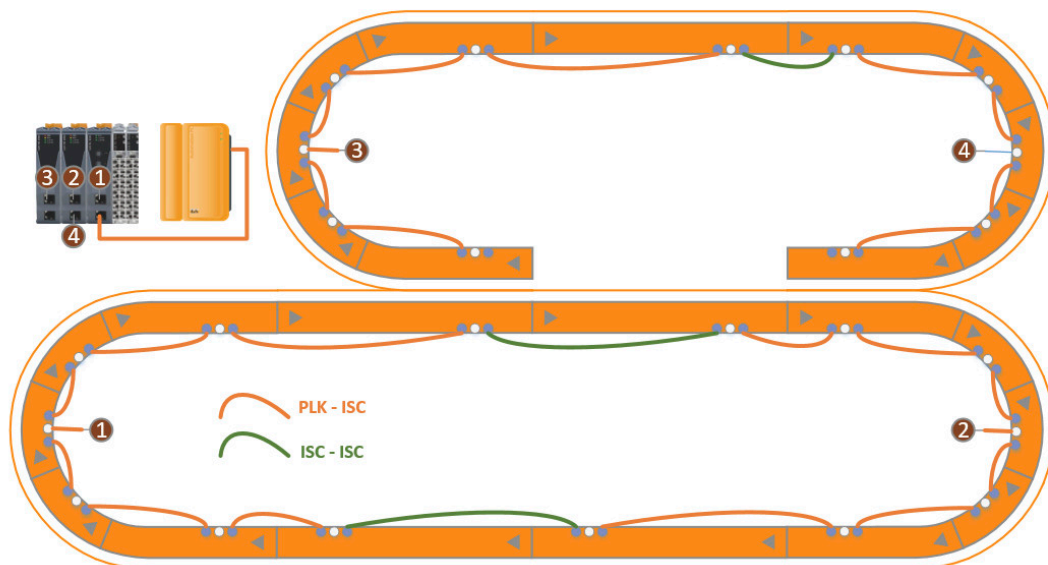


Figure 6: Possible POWERLINK distribution

Guidelines

- The POWERLINK cable coming from the controller must be connected to the center POWERLINK port of the segment.
- The number of hub levels on a POWERLINK string is not permitted to exceed 10.
- Adjacent and opposite-facing segments are permitted to have a maximum hub level difference of ± 2 .
- The POWERLINK ring is not permitted to be closed, i.e. the various POWERLINK strands are not permitted to be connected.
- POWERLINK can be connected to adjacent segments, but not to opposite-facing segments.



Communication / POWERLINK / Wiring / Hubs and jitter



For an explanation of the effect that a closed POWERLINK ring has on the system, see TM1423 in chapter "Diagnostics".

Dynamic node allocation (DNA)

In order that the node number does not have to be set separately for each segment, it is possible to use the DNA setting in the POWERLINK configuration of the segments.

Therefore the DNA parameter has to be activated with the value "On".

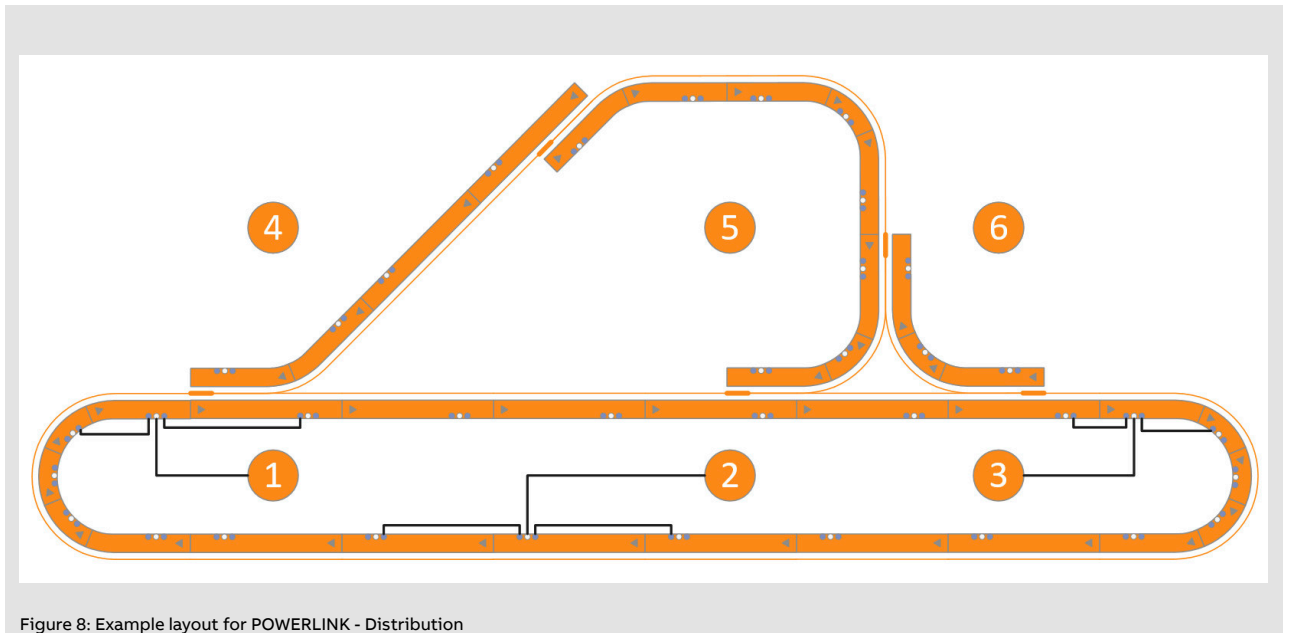
Name	Value
8F1I01.AA66.xxxx-1	
POWERLINK configuration	
POWERLINK parameters	
Dynamic Node Allocation	
DNA	On

Figure 7: Configuration of DNA in a segment

Exercise: POWERLINK - Distribution

To enable the controller to communicate with the segments, they must be connected via POWERLINK. In order for communication via POWERLINK and also between the segments to function well, the hub levels of neighboring and opposite segments are not permitted to differ by more than ± 2 . The following steps must be performed in this exercise:

- 1) Connect the missing connection points 4, 5 and 6 to the middle POWERLINK port of the respective segment.
- 2) Run POWERLINK to the neighboring segments in both directions.
- 3) Enter the hub level next to the segments and check the maximum difference of ± 2 .



3 Initial project with ACOPOSTrak

Once a layout has been planned and the distribution of POWERLINK has been determined, the project can be created in Automation Studio. First, the hardware is added to the project and configured. The assembly is then compiled from the individual segments using a further configuration.

Getting started

The chapter "Getting started ACOPOSTrak" in Automation Help explains how to add and configure the hardware correctly. A mechatronic design with the required configurations and two program tasks can be used for fast execution of an initial movement.

Exercise: "Getting started"

In this exercise, participants will use Automation Help to learn how to create an initial project, add the required hardware and configure it correctly. In addition, participants will also become familiar with the first steps required for the initial movement of the shuttles.

- 1) Start Automation Studio.
- 2) Open Automation Help.
- 3) Open the "Getting started" tutorial in ACOPOSTrak and follow the defined steps.
- 4) Use the Watch window to switch on the track and set the shuttles in motion.
- 5) Observe the behavior via the Scene Viewer.



Drive technology / mapp Motion / Guides / "Getting started" / ACOPOSTrak

3.1 Term definitions

The following briefly summarizes and explains the most important terms for ACOPOSTrak. For detailed documentation, see Automation Help.

Terms

- **Segment:**
Hardware component and heart of ACOPOSTrak (motor module with integrated drive).
- **Track:**
Placing segments in sequence
- **Assembly:**
Compilation of tracks
- **Shuttle:**
Mobile, passive part of ACOPOSTrak. Treated almost like a single axis
- **Assembly configuration:**
A logical component that describes the overall system and contains information about the alignment of the tracks.
- **Shuttle stereotype:**
Contains the shuttle-specific parameters
- **Sector:**
Defined sub-area in the workspace. It can be open or closed and can also continue over multiple tracks.
- **Process point:**
Defined point on a sector with various functions
- **Assembly feature:**
Various functions of an assembly, e.g. couplings, are enabled with this configuration.

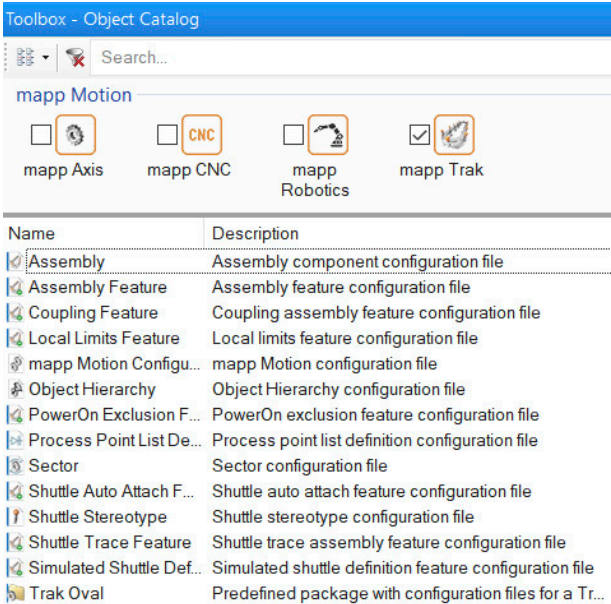


Drive technology / mapp Motion / Concept / mapp Motion / mapp Motion components
 Drive technology / mapp Motion / Concept / Drive technology / Components of a drive system
 Drive technology / mapp Motion / Configuration / Basic components

3.2 Configuration of ACOPOSTrak

Multiple files are available in the Toolbox for configuring ACOPOSTrak.

The mapp Trak filter displays all available configurations. These configuration files must be added to the mapp Motion folder in the Configuration View with a double-click or via drag-and-drop.



Segment

Hardware and configuration are linked via segment variables. These are assigned to the segments and automatically declared in the background as global variables of the data type "McSegmentType". This variable can then be used to uniquely address the corresponding segment.

Assembly configuration

Here, the track segments are structured from the individual segments. The track segments are then aligned and positioned to form the assembly.

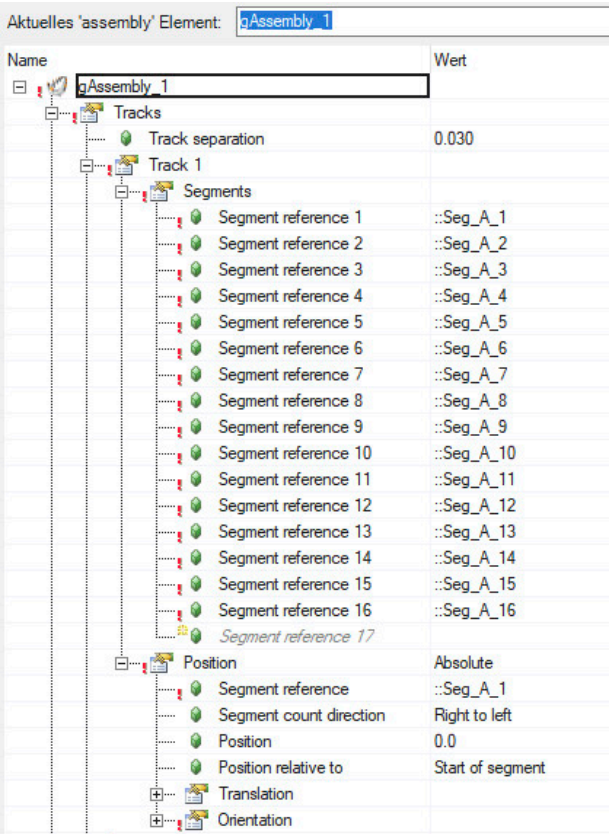


Figure 9: Configuration of an assembly

Shuttle stereotype configuration

The maximum permissible motion parameters, user data size and initial shuttle size are set in the shuttle stereotype configuration. To come up with the correct gear ratio, only the measurement resolution is important for couplings. The unit of measurement is set to meters.

The segments are referenced sequentially as defined in the layout sketch. You can start with any segment, but must then continue with the adjacent segments, moving either clockwise or counterclockwise. For the first track segment, the absolute position must be defined. To do this, a segment whose workspace contains the zero point must be selected. The counting direction for the assembly is also defined for the first track segment. The controller parameters are defined as well. It is possible to create multiple parameter sets. At runtime, it is possible to modify them or switch between them. Changes will be applied the next time the controller is switched on. There are more settings available for shuttles, assembly features and HMI. A global variable with the name of the assembly is created in the background for the assembly. This variable is needed to address the assembly via the application.

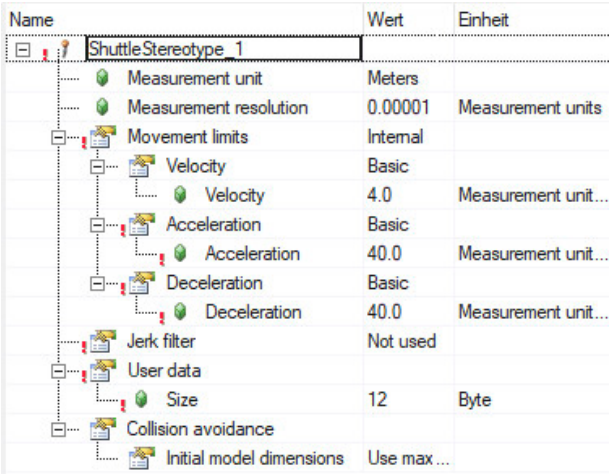


Figure 10: Shuttle stereotype configuration

Sector configuration

Sectors can be defined either by segments or by existing sectors.

When defining via segments, a start and end point must be defined for the respective segments. In addition, all segments that the sector runs on must be entered one after the other, starting at the starting point. This tells us the length and direction of the sector.

When defining an existing sector, only the starting point at the reference sector, the direction and the length have to be specified.

	Sektor_Prozess1	
	Type	Composed sector
	Start segment	
	Segment reference	::Seg_A_1
	Position	0.125
	Position relative to	Start of segment
	Intermediate segments	
	Segment reference 1	::Seg_A_2
	Segment reference 2	
	End segment	
	Segment reference	::Seg_A_3
	Position	0.25
	Position relative to	End of segment
	Sektor_Prozess2	
	Type	Sub sector
	Sector reference	Sektor_Wartung
	Start	
	Position	0.5
	Position relative to	Start of sector
	Length	1.25
	Direction	Positive

Figure 11: Configuring a sector

3.3 Enabling ACOPOStrak in the simulation

Programming a step sequencer is recommended for switching on the assembly with one command, receiving all shuttle references and executing an initial movement with all shuttles.

This section lists and explains the steps that are used in "Getting started".

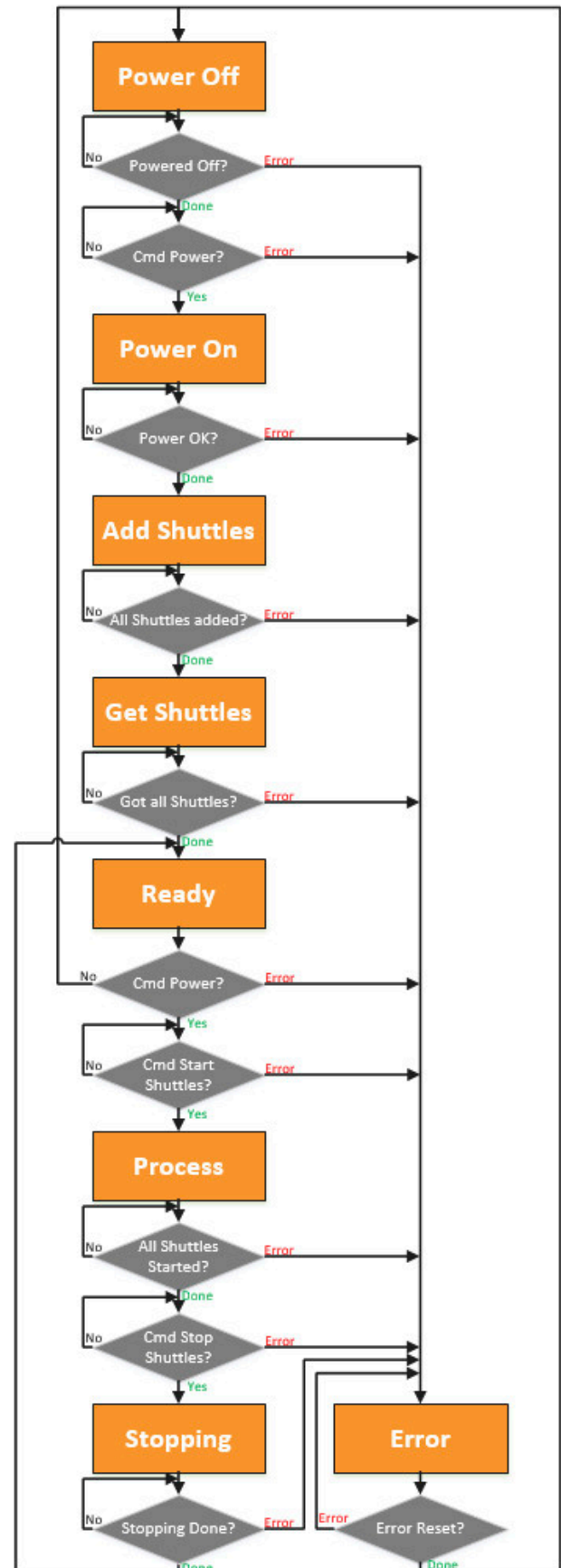


Figure 12: Diagram of the step sequencer used

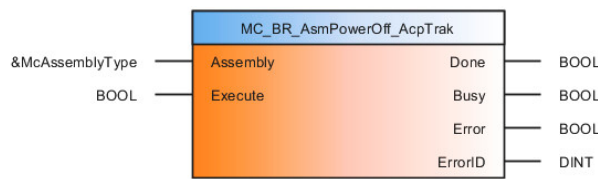


Figure 13: Function block MC_BR_AsmPowerOff_AcpTrak



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_AsmPowerOff_AcpTrak

Power on:

After using the "Power" command, all segments and controllers are switched on. As soon as the "Done" output of the function block is TRUE, the next step is jumped to.

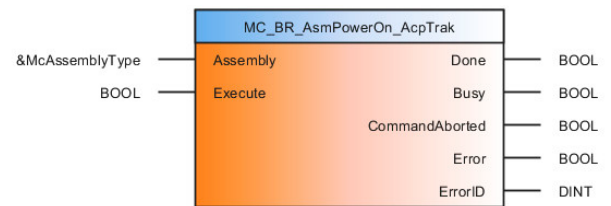


Figure 14: Function block MC_BR_AsmPowerOn_AcpTrak



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_AsmPowerOn_AcpTrak

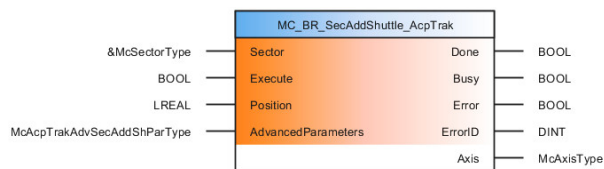


Figure 15: Function block MC_BR_SecAddShuttle_AcpTrak

Add shuttle:

In order to be able to work with shuttles in the simulation, they must first be added to the system. The function block is used to add shuttles to the system on a reference sector at the specified positions. As soon as the "Done" output is TRUE when adding the last shuttle, the next step is jumped to.

This step can alternatively be replaced by the assembly feature "Simulated Shuttles".



Figure 16: Position of the shuttles for the simulation



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_SecAddShuttle_AcpTrak

Get shuttle:

In this step, the axis references of all shuttles are queried and stored in an array so that global movement commands can be executed. As soon as the "Done" output is TRUE and "Remaining count" is 0, the assembly is ready to start movements and jumps to the "Ready" step.

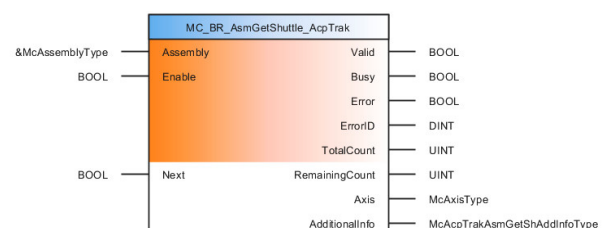


Figure 17: Function block MC_BR_AsmGetShuttle_AcpTrak

?

Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_AsmGetShuttle_AcpTrak

Ready:

In this step, the assembly is ready for use and waits for commands from the user. All shuttles can be started with the "ShuttleStart" command in the **gTrakAsm** structure.

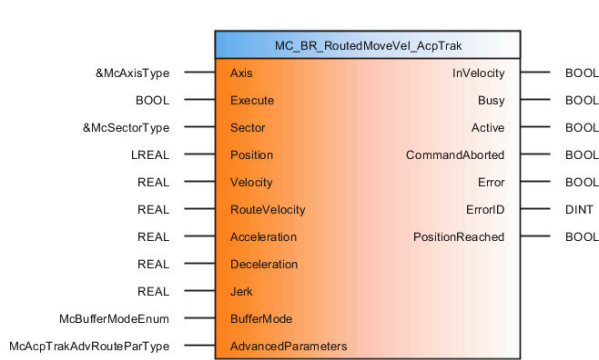


Figure 18: Function block MC_BR_RoutedMoveVel_AcpTrak

Process:

The command in the "Ready" step initiates a jump to the "Process" step, where all shuttles are set in motion. After all shuttles are in motion, they can be stopped again with the command "ShuttleStop" in the "gTrakAsm" structure.

?

Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_RoutedMoveVel_AcpTrak

Stopping:

This step is enabled by the "ShuttleStop" command and all shuttles are brought to a standstill one after the other. As soon as all shuttles are stopped, the system jumps back to the "Ready" step.

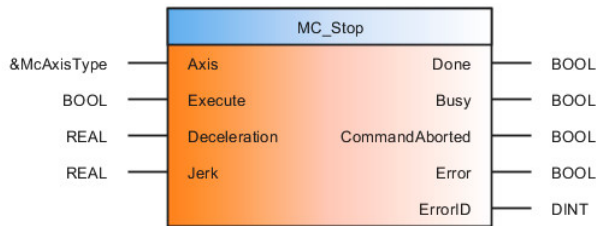


Figure 19: Function block MC_Stop

?

Drive technology / mapp Motion / Libraries / Core / McAxis / Function blocks / MC_Stop

3.4 HMI application with Scene Viewer

The Scene Viewer can make the behavior of ACOPOStrak visible and the application can also be properly tested in the simulation. To connect the Scene Viewer to the application, a task is required, which can be found in the help documentation.

File device

A file device with the name **SvgData** can be created in the CPU configuration so that the Scene Viewer files can be generated after installation. The path to which the file device refers must already exist. The files are generated automatically after the controller or simulation has been started.

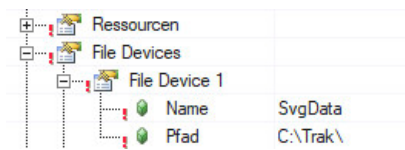


Figure 20: File device configuration

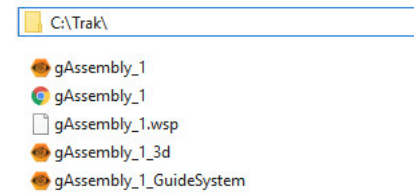


Figure 21: Files created for Scene Viewer

HMI task

In this task, the information about the shuttles is written to a structure that is accessed by the Scene Viewer. The sample task can be found in the help documentation and can be changed.

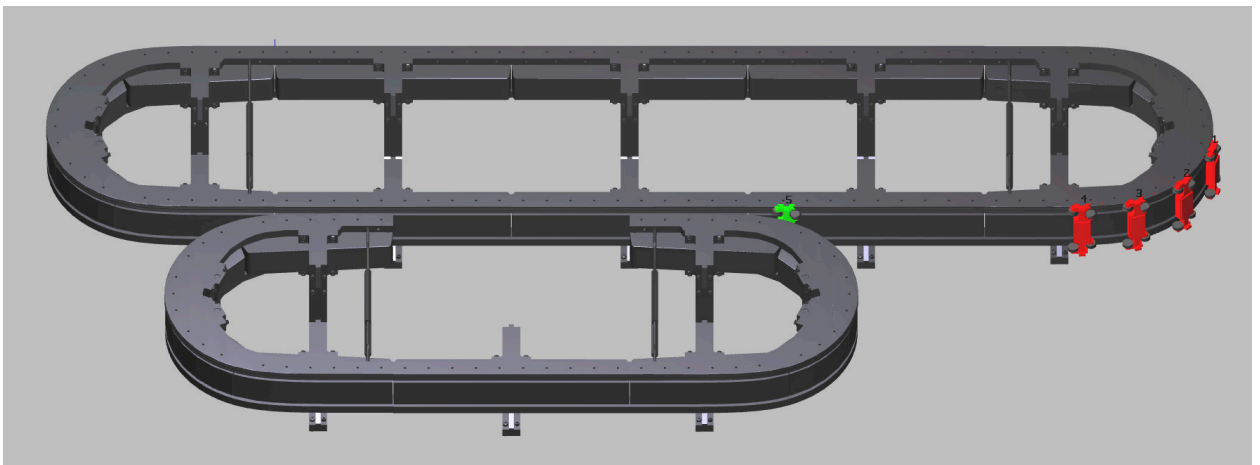
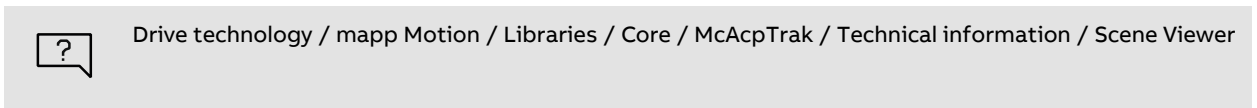


Figure 22: HMI application with Scene Viewer in 3D

4 ACOPStrak electrical dimensioning

This chapter explains the design of the electrical power supply.

4.1 Sizing in the simulation

The entire electrical dimensioning process can be carried out in the simulation and should always be integrated into creation of the application. This means that heavily burdened segments can be identified at an early stage and the processes can be adapted.

The simulation provides real values since the power consumed in the segments can be calculated for each shuttle based on the movement parameters.

Dimensioning task

A task is currently available for dimensioning the power supply, which can be easily imported into the application.

This task cyclically calculates the required power in the background. In addition to the average values, peak values and the utilization of the segments at different ambient temperatures are also determined. The calculation is based on the number of switched on segments, the number of shuttles and their movement profile, which is constantly read by the task.

The results from this task can then be saved to a CSV file.

4.2 Power supply structure

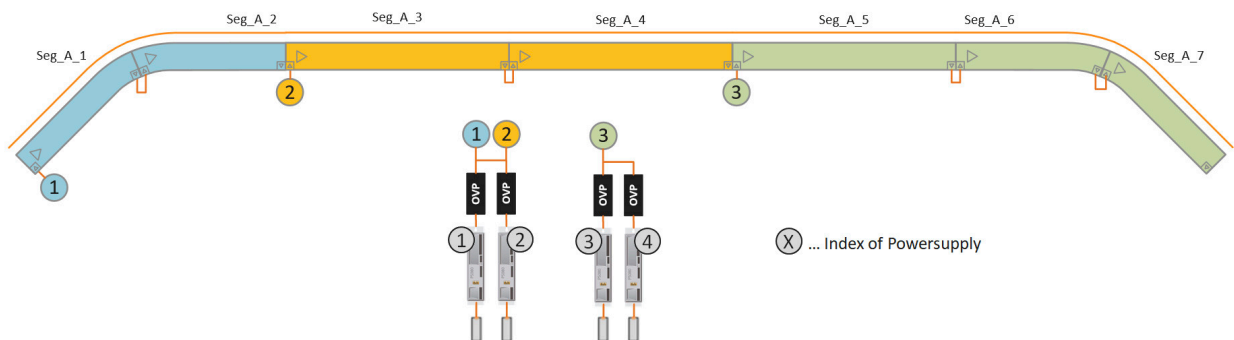


Figure 23: Power supply example

The power supply consists of multiple sections:

- Power supplies:**
 The power supplies supply the assembly with power. To increase the available power, 2 power supplies can be connected in parallel. For safety reasons, each power supply is equipped with overvoltage protection downstream, which also limits the voltage to 60 V via the hardware. A braking resistor must also be installed for each power supply. This converts the excess energy that is fed back into the system by the shuttles and not required by other consumers into heat.
- Cables:**
 The cables connect the power supply modules to the segments. Multiple cables can be connected to one supply module.
- Segments:**
 The segments have a power supply input and output. This means that multiple segments can be supplied via one cable. It is important to note that connected segments do not require more power than the amount permitted to be transported via the cable.

4.3 Configuration

The configuration of the dimensioning task takes place in the INIT file for the task and is divided into multiple sections:

- Maximum number of components
- General parameters
- Product parameters
- Segments parameters
- Cable parameters
- Power supply parameters
- Online interaction

The images of the configuration are based on the figure below [Fig. 23 "Power supply example" on page 19.](#)

Maximum number of components

In the global variable file **gTrakSupport.var**, which is added with the dimensioning task, there are 4 constants that determine the maximum number of segments, shuttles, cables and power supplies. Since the size of the arrays for the variables and function blocks is specified via these constants, the values should be adapted as precisely as possible to the actual number in order to avoid an unnecessary increase of the CPU load.

In addition, this variable file contains the variable gTrakDesignShuttleOnline, which is explained in section ["Online interaction parameters"](#).

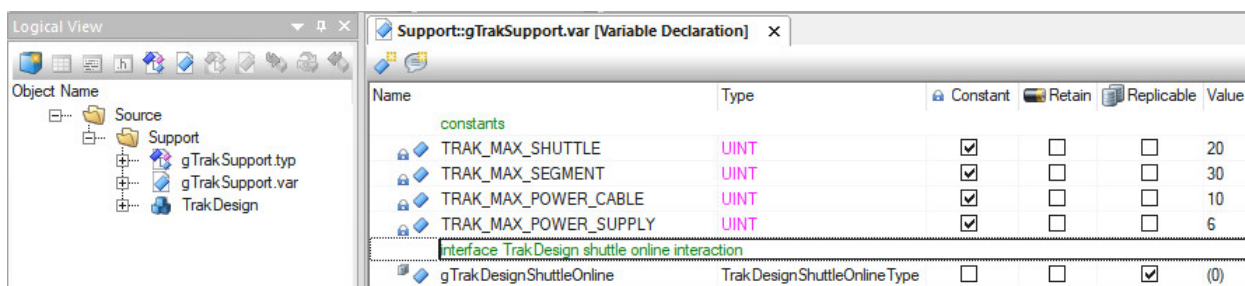


Figure 24: Variable declaration of constants



If the variable file is not available, it must be created later, and the 4 constants must be declared according to the figure [Fig. 24 "Variable declaration of constants"](#).

General parameters

The general parameters are used to configure the data storage and to specify the assembly reference. In addition, the type of shuttle used and the ambient temperature are specified.

```
// general parameter
TrakDesign.AssemblyRef      := ADR(gAssembly_1);
TrakDesign.AssemblyName    := 'gAssembly_1';
TrakDesign.EnableCalc      := TRUE;
TrakDesign.CmdReset        := FALSE;
TrakDesign.CmdSaveReport   := FALSE;
TrakDesign.CmdRecordDiag   := FALSE;

TrakDesign.ParaShuttleType  := 100;
TrakDesign.ParaTempAmbient := 25;

// TrakDesign.ParaMechShuttleIx := 1;
TrakDesign.ParaFileDevice   := 'User';
TrakDesign.ParaFileNameReport := 'TrakDesignReport.csv';
TrakDesign.ParaFileNameDiag := 'TrakDesignDiag.dat';
TrakDesign.ParaFileCsvStyle := 1;
TrakDesign.ParaDiagPrescale := 10;
```

Figure 25: General parameters in INIT

Shuttle payload parameters

These parameters are used to configure a product holder, up to 8 products and external forces. It is possible to change between the products and the externally acting forces at runtime (see ["Online interaction parameters"](#)). This enables you to carry out a realistic simulation of power consumption and to dimension the power supply accordingly.

• Product holder:

The weight and center of gravity of the product holder relative to the shuttle center of gravity is configured here. If known, the mass moment of inertia of the product holder can also be specified for an even more precise simulation. Otherwise, this parameter must be specified as 0.0.

```

// shuttle payload parameter
// carrier
TrakDesign.Carrier.ParaMass           := 0.0;           // [kg]
TrakDesign.Carrier.ParaMassCenterX    := 0.0;           // [m]
TrakDesign.Carrier.ParaMassCenterY    := 0.0;           // [m]
TrakDesign.Carrier.ParaMassCenterZ    := 0.0;           // [m]
TrakDesign.Carrier.ParaMassInertiaZZ := 0.0;           // [kg m^2]

```

Figure 26: Product holder parameters

- **Product:**

The weight and center of mass of up to 8 different products relative to the shuttle center of gravity is configured here. If known, the mass moment of inertia of the product can also be specified for an even more precise simulation. Otherwise, this parameter must be specified as 0.0.

A separate name can be assigned to each product.

```

// shuttle payload parameter
// product
TrakDesign.Payload[1].ParaName        := 'no product on carrier';
TrakDesign.Payload[1].ParaMass        := 0.0;           // [kg]
TrakDesign.Payload[1].ParaMassCenterX := 0.0;           // [m]
TrakDesign.Payload[1].ParaMassCenterY := 0.0;           // [m]
TrakDesign.Payload[1].ParaMassCenterZ := 0.0;           // [m]
TrakDesign.Payload[1].ParaMassInertiaZZ := 0.0;         // [kg m^2]

```

Figure 27: Product parameters

- **External force:**

Up to 8 different external forces can be configured here. The X, Y and Z components of the force and the coordinates of the point of application relative to the shuttle center are specified for this purpose.

An external force can, for example, brake a shuttle. The controller attempts to compensate for this delay, which in turn results in higher power consumption.

```

// shuttle payload parameter
// external force
TrakDesign.Payload[1].ParaExtForceX   := 0.0;           // [N]
TrakDesign.Payload[1].ParaExtForceY   := 0.0;           // [N]
TrakDesign.Payload[1].ParaExtForceZ   := 0.0;           // [N]
TrakDesign.Payload[1].ParaExtForceAppPointX := 0.0;       // [m]
TrakDesign.Payload[1].ParaExtForceAppPointY := 0.0;       // [m]
TrakDesign.Payload[1].ParaExtForceAppPointZ := 0.0;       // [m]

```

Figure 28: External force parameters

- **Initial assignment:**

During initial assignment, any configured load (product + external force) can be assigned to a shuttle.

However, the assignment can also be made automatically via the current controller parameter set. The index for the load is changed at the same time as the index for the controller parameter set.

```

FOR i:=1 TO TRAK_MAX_SHUTTLE DO
    TrakDesign.Shuttle[i].ParaPayloadIx := 1;
//    TrakDesign.Shuttle[i].ParaAutoPayload := 0;
//    TrakDesign.Shuttle[i].ParaInitialOrientationFlip := 1;
END_FOR

```

Figure 29: Initial parameter assignment

Segments parameters

For each segment there are 3 parameters that must be set in the INIT file of the task.

- **Name:**

The name of the segment variable that was assigned to the respective segment is entered here. This allows the data to be assigned to the correct segment.

```
// electrical segment assembly
TrakDesign.Segment[1].ParaName := 'Seg_01';
TrakDesign.Segment[2].ParaName := 'Seg_02';
TrakDesign.Segment[3].ParaName := 'Seg_03';
TrakDesign.Segment[4].ParaName := 'Seg_04';
TrakDesign.Segment[5].ParaName := 'Seg_05';
TrakDesign.Segment[6].ParaName := 'Seg_06';
TrakDesign.Segment[7].ParaName := 'Seg_07';
```

Figure 30: Segment parameter name

- **Type:**

This parameter defines which segment types are involved, e.g. straight lined, curved or circular arc segments.

```
// electrical segment assembly
TrakDesign.Segment[1].ParaType := TYPE_AB;
TrakDesign.Segment[2].ParaType := TYPE_BA;
TrakDesign.Segment[3].ParaType := TYPE_AA;
TrakDesign.Segment[4].ParaType := TYPE_AA;
TrakDesign.Segment[5].ParaType := TYPE_AA;
TrakDesign.Segment[6].ParaType := TYPE_AB;
TrakDesign.Segment[7].ParaType := TYPE_BA;
```

Figure 31: Segment parameter type

- **Cable index:**

The cable index indicates which cable supplies the segment. If the power supply is daisy-chained through to additional segments, the cable index remains the same.

```
// electrical segment assembly
TrakDesign.Segment[1].ParaIdxCable := 1;
TrakDesign.Segment[2].ParaIdxCable := 1;
TrakDesign.Segment[3].ParaIdxCable := 2;
TrakDesign.Segment[4].ParaIdxCable := 2;
TrakDesign.Segment[5].ParaIdxCable := 3;
TrakDesign.Segment[6].ParaIdxCable := 3;
TrakDesign.Segment[7].ParaIdxCable := 3;
```

Figure 32: Segment parameter cable index

Cable parameters

This setting specifies the length of each cable and the power supply that the power is drawn from. If two power supplies are connected in parallel, one of the two power supplies can be specified.

```
// electrical power cable assembly
TrakDesign.Cable[1].ParaLength := 10.0;
TrakDesign.Cable[2].ParaLength := 10.0;
TrakDesign.Cable[3].ParaLength := 10.0;
TrakDesign.Cable[1].ParaIdxSupply := 1;
TrakDesign.Cable[2].ParaIdxSupply := 1;
TrakDesign.Cable[3].ParaIdxSupply := 3;
```

Figure 33: Cable parameters

Power supply parameters

The type of power supply must be specified for the power supply parameters.

If two power supplies are connected in parallel, this must be configured here via the power supply indices. This configuration is only done for the power supply with the lower index. The other power supply unit is assigned 0 as a parameter.

```
// electrical supply assembly
TrakDesign.Supply[1].ParaType := PS080;
TrakDesign.Supply[2].ParaType := PS080;
TrakDesign.Supply[3].ParaType := PS080;
TrakDesign.Supply[4].ParaType := PS080;
TrakDesign.Supply[1].ParaIdxSupplyParallel := 2;
TrakDesign.Supply[2].ParaIdxSupplyParallel := 0;
TrakDesign.Supply[3].ParaIdxSupplyParallel := 4;
TrakDesign.Supply[4].ParaIdxSupplyParallel := 0;
```

Figure 34: Power supply parameters

Online interaction parameters

The variable `gTrakDesignShuttleOnline` can be used to influence the electrical dimensioning at runtime.

When the assembly is switched on, all shuttles that are already on the track are automatically detected. If additional shuttles are added during operation, they must be added to the task manually. Here, the shuttle reference and shuttle ID are transferred with the command "CmdAddShuttle".

interface TrakDesign shuttle online interaction

TrakDesignShuttleOnline Type			
CmdSetPayloadIx	BOOL		☐
CmdAddShuttle	BOOL		☐
CmdDeleteShuttle	BOOL		☐
ParaPayloadIx	USINT		☐
ParaShuttleID	UINT		☐
ParaShuttleAxis	McAxisType		☐
Done	BOOL		☐

Figure 35: General parameters in INIT

If the load on the shuttle should be changed at runtime, e.g. after a process, this can be done using the variable "ParaPayloadIx" and the command "CmdSetPayloadIx".

This enables a realistic simulation of the power consumption and its distribution in the system.

Exercise: Electrical dimensioning

In this exercise, you will learn how to electrically design ACOPOSTrak. The dimensioning can be carried out in the simulation since it delivers realistic values. A ready-made program task is available for dimensioning, which can be easily imported and configured. The measured data can then simply be saved as an Excel table.

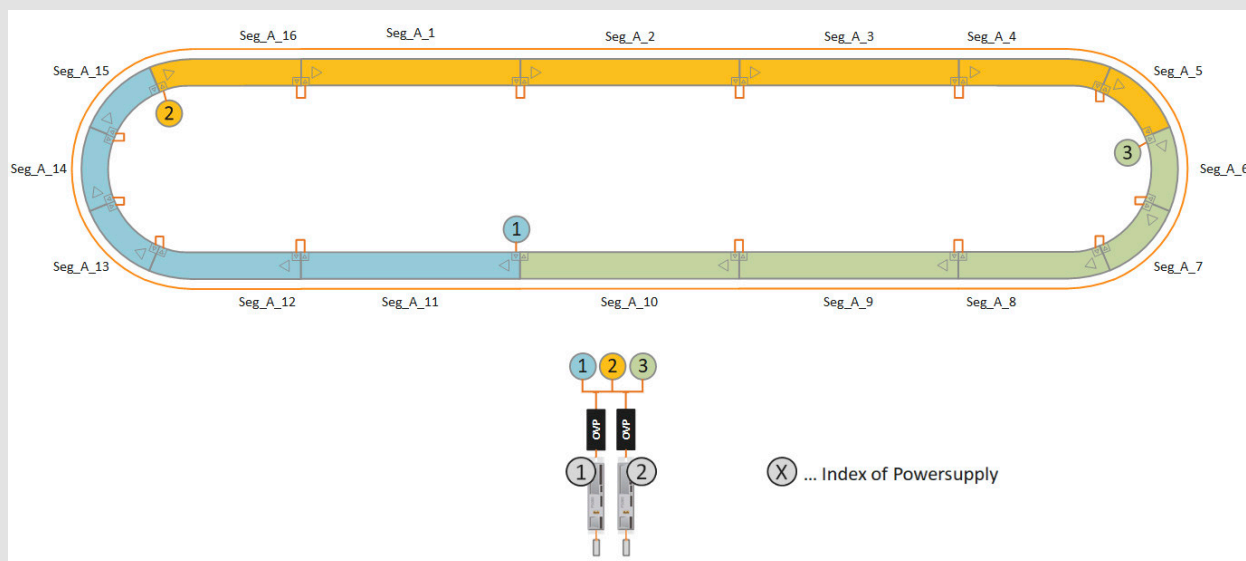


Figure 36: Layout for electrical dimensioning

The following steps are necessary:

- 1) Import the dimensioning task "TrakDesign" into the "Getting started" application and assign it to task class 1.
- 2) Create a new "User" file device.
- 3) Adapt the configuration in the Init of the dimensioning task to the layout shown above.
- 4) Transfer the application, run a test and save the data.
- 5) Evaluate the results and identify possible hotspots.



Explanations of the dimensioning task in the enclosed PDF file.
The "MTBasics" library is required and might have to be added manually.

4.4 Results

The results from the dimensioning task are stored in the configured file device. This file lists the power consumed for each shuttle, segment, cable and power supply.

If the power consumption is too high, warnings are noted for the respective modules. For this reason, it is possible to make a new plan for distributing the power so that an even load for all modules can be achieved.

In addition, information is listed at the beginning of the file regarding the assembly, the software versions used and the configured loads.

The load at the configured ambient temperature is also calculated for the segments. This makes it possible to estimate whether an excessive load causes the segment to overheat during operation. In this case, the movement profile can already be adjusted at this point during the simulation phase, or active cooling of the segment can be considered.

5 Access to user data with process points

This chapter provides a more detailed explanation of process points and user data.

5.1 Process points

Process points are defined points in the workspace. They are added to sectors and can have various functions.

Process point as trigger point

As soon as a shuttle passes a trigger point, an event is triggered. This event contains the shuttle reference and the direction in which the trigger point was passed.

This event can be read via a function block.

Name	Value	Description
ProcessPointList_1		
Process point: PP_Test	Trak point	
Name	PP_Test	Identifier of point
Sector reference	SectorTrackA	Name of the referenced sector component
Position	1.5	Position on the sector
Position relative to	Start of sector	Absolute position from which the position is counted
Trigger event buff...	10	Amount of maximum saved trigger point events
Barrier functionality	Off	Enable barrier functionality for process point
Process point: {./Name}	Trak point	

Figure 37: Configuration of a process point

The process points are configured in a separate file. During configuration, the process point is given a descriptive name, preferably with reference to the process station. A process point is positioned on a defined sector. In addition, the number of buffered events at the process point can be configured.

Process point as barrier

A process point can also be used as a barrier. This function is explained in chapter 9 "Process points as a barrier" on page 34.

5.2 User data

User data is the data that is generated for each shuttle. The basis for this is a user-defined data structure. The user can design this to be as large and extensive as desired.

The user data can contain, for example, information about the product status, product serial number, shuttle information and other process-relevant data.

With a function block and the shuttle reference, the corresponding user data can be read, modified and written to. The data size for a shuttle is specified in bytes in the "Shuttlestereotype" configuration. The required memory is reserved by the maximum permissible number of shuttles in the assembly.

Based on the user data, process and route decisions can then be made and product information can be saved in databases.

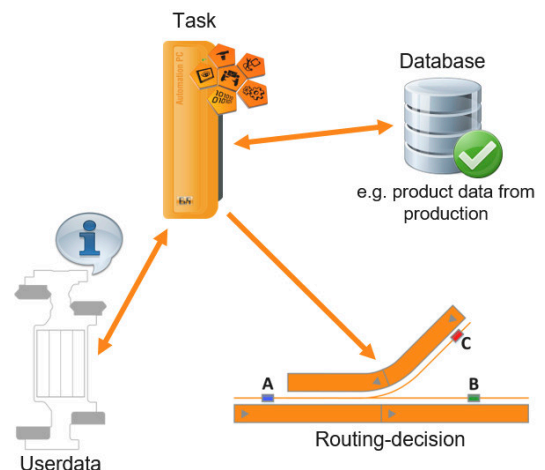


Figure 38: Using the user data

5.3 Integration into the application

Programming in step sequencers is recommended for using process points, especially with regard to their application in process stations. The step sequencer in this chapter forms the foundation for all subsequent process stations. The step sequencer consists of 4 steps; in real applications, more steps can be required in a process station. The individual steps and the function blocks used in them are explained below.



The flowchart of the step sequencer used here can be found in the appendix on page 48.

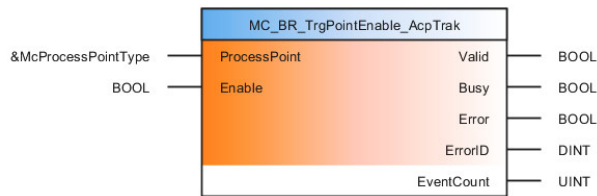


Figure 39: Function block MC_BR_TrgPointEnable_AcpTrak

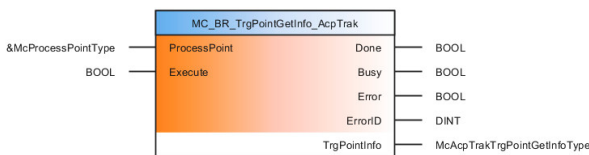


Figure 40: Function block MC_BR_TrgPointGetInfo_AcpTrak

Check trigger

In this step, the process point is activated and the system waits on an event. As soon as an event is triggered at the process point, it is evaluated and the system switches to the next step.

With the function block MC_BR_TrgPointEnable_AcpTrak, the process point is activated, causing the function to be used as trigger point. The number of non-evaluated events is displayed via the "EventCount" output.

An event can be read from the process point with the function block MC_BR_TrgPointGetInfo_AcpTrak. The information from the event contains the shuttle reference and the direction in which the process point was passed.



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_TrgPointEnable_AcpTrak

Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_TrgPointGetInfo_AcpTrak

Get user data

The user data of the shuttle can be copied to a local variable with the shuttle reference from the previous step. This must be from the same data type as the one defined for the user data.

After successfully copying the data, the next step is performed.

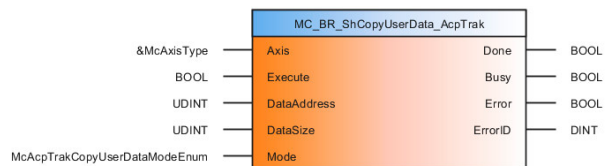


Figure 41: Function block MC_BR_ShCopyUserData_AcpTrak



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_ShCopyUserData_AcpTrak

Process

The process is executed in this step. Based on the user data, decisions for the route and other processes can be made here. When the process is complete or a route has been defined, the next step is performed.

Set user data

In the last step, the modified user data from the local variable is written back to the shuttle. The same function block is used for this as the one in the "Get userdata" step. Only the command at input "Mode" must be changed.

Exercise: Accessing the user data

In this exercise, you will learn how to configure a process point and how to integrate it into the application. You will also learn how to process and access shuttle-related user data.

- 1) Process point - Add configuration file (see [3.2 "Configuration of ACOPOStrak" on page 12](#))
- 2) Configure the new process point "PP_Test" in sector "SectorTrakA" at position 1.5.
- 3) Add the new task "Process_Test".
- 4) Declare the local step variable "Step" with enumerator "Step_enum".

Step_enum:

Step name	Description
CHECK_TRIGGER	Here, an event at the trigger point is waited for and evaluated.
GET_USERDATA	Here, the user data is copied from the shuttle and sent to a local variable.
PROCESS	A process is carried out here and the shuttle is forwarded.
SET_USERDATA	Here, the modified user data is written back to the shuttle.

Table 1: Declaration of steps

- 5) Declare the new local variable "ShuttleUserData" with the data type ShuttleUserData_typ.
- 6) Declare the new local variable "ShuttleAxis" with the data type McAxisType, on which the shuttle reference is then stored.
- 7) A step sequencer should be created for reading the trigger point, reading and writing user data and carrying out a process (for help, see ["Solution: Accessing the user data" on page 45](#)).
- 8) The "Execute" inputs are always reset after a positive edge at the "Done" output.
- 9) As a process, green shuttles should be colored red as they pass by (ShuttleUserData.Color := RED); all others should be colored green (ShuttleUserData.Color := GREEN).
- 10) Test the application in Scene Viewer.

6 Modular application using process stations

Process stations are the work areas in which the shuttles interact. They basically consist of a sector and one or more process points.

In the application, it is recommended to create a separate task or action for each process station. This helps to maintain an overview and allows the program code of a process station to simply be copied and reused for a new process station.

6.1 Concept of the process station

The foundation of a process station is a sector that is the same size as the process station. There is at least 1 process point on it that supplies the shuttle reference for the shuttle to be processed. If required, multiple process points can also be located in the sector.

A process station can be divided into 2 areas. One is the process area, where machining and other operations take place; the other is the waiting area, where shuttles wait for the process area to become free.

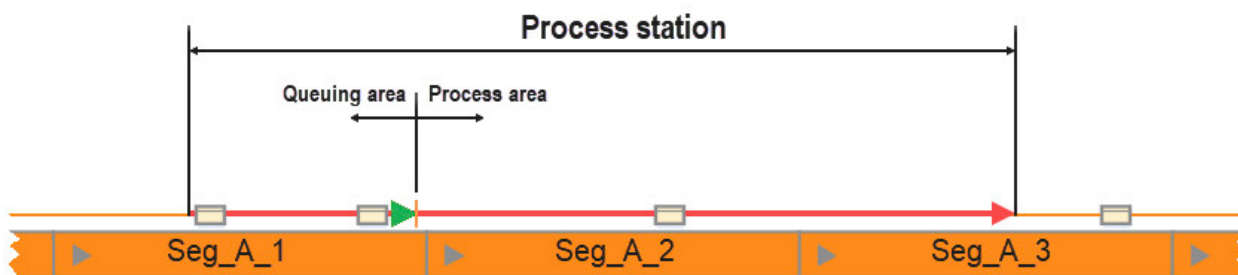


Figure 42: Construction of a process station

Process stations also offer the advantage that the sector and thus the entire process station can be moved very easily. Since the process points and the positions in the process station are both referenced to the sector, no further changes need to be made. The route calculation is also adjusted automatically, since the target position is referenced to the target sector.

6.2 Programming a process station

For programming, we recommend using a process station to create a new task. This way it can be simply copied and modified for any additional process stations. In the task, the process station should be programmed as a step sequencer since this promotes clarity and simplicity.

The step sequencer used for evaluating a process point was already explained in the previous chapter. The same step sequencer is used for the process station.

Exercise: Developing a process station

In this exercise, you will learn how to configure a sector and how to define a process station with it.

- 1) Configure 2 new sectors: "Sector_GetProduct" and "Sector_ReleaseProduct"

Sector name	Beginning of sector			End of sector		
Sector_GetProduct	Seg_A_8	0.3	End of segment	Seg_A_9	0.3	End of segment
Sector_ReleaseProduct	Seg_A_2	0.33	Start of segment	Seg_A_3	0.33	Start of segment

Table 2: Sector definitions

- 2) Rename the process point "PP_Test" to "PP_GetProduct" and enter "Sector_GetProduct" with position 0.1 as the new reference sector.

- 3) Add another "PP_ReleaseProduct" process point at position 0.1 of the "Sector_ReleaseProduct".
- 4) Rename the task "Process_Test" to "Process_GetProduct" and update the process point references to "PP_GetProduct".
- 5) Now, a "Routed move velocity" should be used in the step "Process".
- 6) In "Process_GetProduct", the shuttle should be colored green (`ShuttleUserData.Color := GREEN`) and sent with 1 m/s to position 0.0 at the target sector "Sector_ReleaseProduct".
- 7) Copy and paste the entire process task and rename it to "Process_ReleaseProduct".
- 8) Update the process point references; the shuttle is now colored red (`ShuttleUserData.Color := RED`) and returned with 2 m/s to position 0.0 at the target sector "Sector_GetProduct".
- 9) The "Execute" input of the "RoutedMove" function block is reset after a positive edge on the "Active" output.
- 10) Observe the processes in the Scene Viewer.

7 Enhancing a layout

Due to the flexible design of ACOPOSTrak, it is also possible to extend later on it with minimal effort. Segments can be built into an existing track and additional tracks can be added.

7.1 Adjustments in System Designer

In the System Designer, the segments are added, arranged and connected with POWERLINK. Segment variables are assigned to the new segments again.

If an existing track should be extended, it is recommended to separate the segments that are not connected to POWERLINK. This way, the configuration of the segments does not need to be changed.

7.2 Configuration of the additional segments

If an existing track is extended, the new segments in the assembly configuration must be inserted at the correct position in the list for the corresponding track.

When a new track segment is added, it is created in the same way as the existing tracks. There are 2 positioning options:

- **Relative to one segment:** A position is specified on one segment of the existing track and one segment of the new track.
- **Relative to two segments:** Two pairs of segments are specified, at which the new track is positioned.

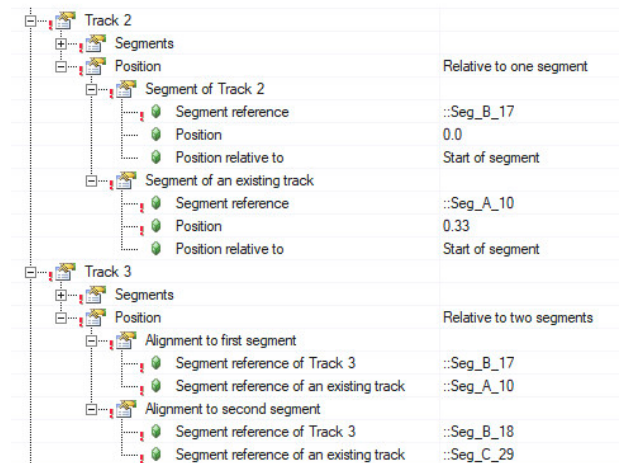


Figure 43: Configuration of further tracks and positioning options

Exercise: Extending the assembly with diverters

In this exercise, a second track is added to the existing track. You will learn how to configure a diverter in the assembly configuration.

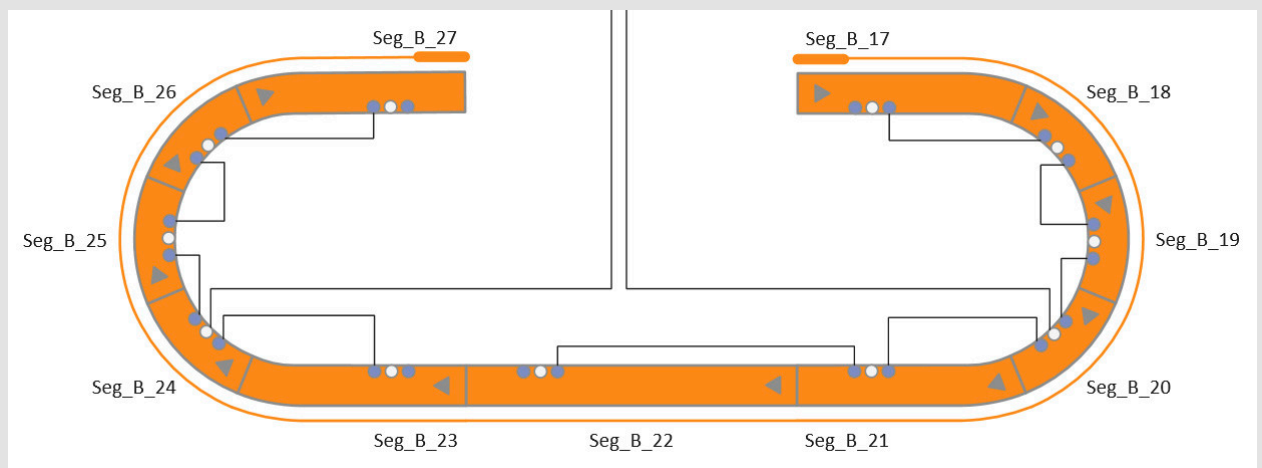


Figure 44: Layout for the second oval

The following steps are required:

- 1) Add the required hardware and connect it with POWERLINK.
- 2) Assign the segment names and variables (see figure 27: Layout for second oval)
- 3) Set the POWERLINK node numbers and configure the DNA.
- 4) Logically structure the track in the assembly configuration.
- 5) Position the second track relative to a segment on the existing track.

The Seg_B_17 segment should be positioned at position 0.33 on the Seg_A_10 segment.

8 Types of movements

There are 2 types of movements that allow shuttles to drive on the track. There are elastic movements and rigid movements.

The target position can be specified as absolute or relative for both movement types.



Drive technology / mapp Motion / Concept / Control / Motion control / Elastic and rigid movements

8.1 Elastic movements

The objective of an elastic movement is to allow a shuttle to reach the target position despite temporary obstacles such as barriers and slower moving shuttles. The shuttles can stop or reduce their speed. This behavior is required for switches since 2 paths are combined to one here.

Route movements are a subtype of elastic movements. This type is used to switch between the reference sectors of the shuttles and therefore also between the tracks.

The movement type is shown in the function block name. This means that function blocks for elastic movements on the same reference sector have "ElasticMove..." in their name and function blocks for route movements have "Routed-Move...".

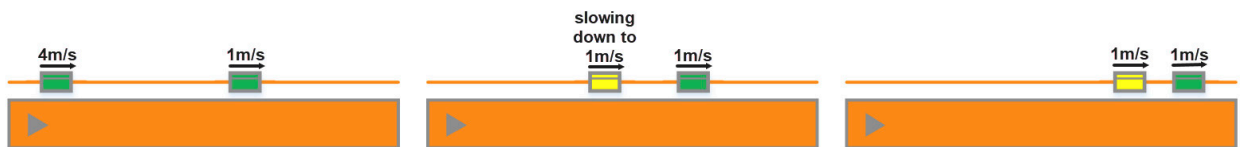


Figure 45: Shuttle behavior when there is an obstacle with elastic movement



For a more detailed description of the available function blocks, see the help documentation:
Drive technology / mapp Motion / Libraries / Core / McAcTrak / Function blocks

8.2 Rigid movements

The objective of a rigid movement is to execute the movement with the specified parameters. If this is not possible due to an obstacle, the shuttle stops with an error that must be acknowledged before starting a new movement.

Rigid movements also include couplings. These are covered in chapter 10 "ACOPOstrak couplings" on page 35.

Function blocks from the "McAxis" library are available for rigid movements.

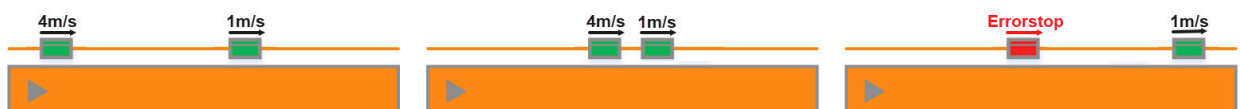


Figure 46: Shuttle behavior when there is an obstacle with rigid movement



For a more detailed description of the available function blocks, see the help documentation:
Drive Technology / mapp Motion / Libraries / Core / McAcTrak / Function blocks / Supported function blocks of the "McAxis" library

Exercise: Process station with rigid movement

In this exercise, an additional process station with 2 process points is defined on the newly added track. A rigid movement should be performed in the process station.

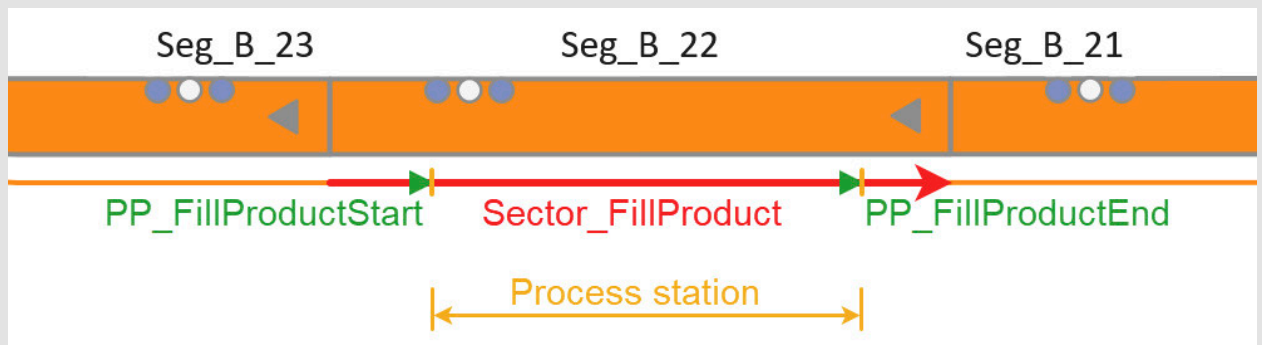


Figure 47: Structure of the process station "Process_FillProduct" with 2 process points

- 1) Create another "Sector_FillProduct" sector.

Sector name	Beginning of sector			End of sector		
Sector_FillProduct	Seg_B_22	0.0	End of segment	Seg_B_22	0.0	Start of segment

Table 3: Sector definition

- 2) Add the 2 process points "PP_FillProductStart" and "PP_FillProductEnd" to the sector "Sector_FillProduct" (0.1 m from the start and end of a sector).
- 3) The target sector in the process "Process_GetProduct" changes from "Sector_ReleaseProduct" to "Sector_FillProduct".
- 4) Copy the task "Process_GetProduct" and rename it to "Process_FillProduct_End".
- 5) Adapt the step sequencer to the new task.
- 6) Copy the task "Process_FillProduct_End" and rename it to "Process_FillProduct_Start".
- 7) The "RoutedMove" function block in "Process_FillProduct_End" is replaced with the "MC_MoveVelocity" function block (rigid movement).
- 8) The shuttle should move at 0.5 m/s from the beginning of the process station to the process point at the end of the station and will be colored orange. (ShuttleUserData_FillStart.Color := ORANGE)
- 9) The target sector for the "RoutedMove" function block in "Process_FillProduct_End" is "Sector_ReleaseProduct". The shuttle should move to position 0.0 at 1 m/s and will be colored blue (ShuttleUserData_FillEnd.Color := BLUE).

9 Process points as a barrier

In addition to their function as trigger points, process points can also serve as barriers. These can be controlled via the application.

9.1 Types of obstacles

There are two types of obstacles in an assembly. There are permanent obstacles that a shuttle cannot overcome at any time and there are temporary obstacles that can be overcome by using an elastic movement.

Permanent obstacles

These obstacles can never be overcome by shuttles. If shuttles reach this type of obstacle, they stop with an ErrorStop. Permanent obstacles can be the end of a sector or oncoming shuttles.

Temporary obstacles

These obstacles can be overcome by shuttles. The shuttles make a normal stop in front of the obstacle and resume movement after the obstacle disappears. Temporary obstacles include barriers and slower moving shuttles.



Drive technology / mapp Motion / Concept / Control / Motion control / Elastic and rigid movement / Obstacles and process points

9.2 Barriers

As previously described, barriers are temporary obstacles and can be crossed by shuttles.

The basis for a barrier is a process point. Using the function block MC_BR_BarrierCommand_AcpTrak, the barrier can be controlled by the application. There are 3 commands, two of which are for opening and closing the barrier manually. The third command activates the ticket system, which makes it possible to only allow a specified number of shuttles to pass.

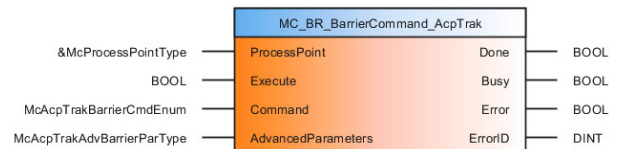


Figure 48: Function block MC_BR_BarrierCommand_AcpTrak



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_BarrierCommand_AcpTrak

Exercise: Process point as barrier

In this exercise, you will learn how to use a process point as a barrier. If the command variable has a positive edge, 2 shuttles should enter the process station.

- 1) Open the task "ProcessFillProduct_Start".
- 2) Declare a "cmdFillStart" command variable with the data type BOOL.
- 3) Declare the required function block for using the barrier.
- 4) "PP_FillStart" is used as the process point and the ticket count is set to 2.
- 5) Enable the ticket system using the command "mcACPTRAK_BARRIER_ADD_TICKETS".
- 6) If "cmdFillStart" is True, the "Execute" input of the barrier is set and "cmdFillStart" is reset.
- 7) After a positive edge at the "Done" output of the barrier, the "Execute" input is reset again.



The barrier functionality must be enabled in the process point configuration.

10 ACOPOStrak couplings

Couplings expand the options with ACOPOStrak. For one, a load can be divided among multiple shuttles. There is also the option of using synchronized movements, e.g. when transferring to star wheels.

10.1 General information

A coupling is a rigid movement in which the setpoint is not generated based on a calculated route, but rather on the current movement parameters and position of the coupling master. Any type of axis can be used as the master of a coupling, such as a single axis or another shuttle. These two possibilities will be discussed in more detail below.

Coupling in the software also allows shuttles be coupled mechanically, which increases the payload because it can be divided among the individual shuttles.

10.2 Configuring a coupling object

In order to couple a shuttle to an axis, a coupling object must first be added to the shuttle. The coupling object activates a central setpoint generator for the shuttle, which also has the cam functionality. A coupling object is an assembly feature and is configured in the assembly feature file.

Name	Wert
Feature_Coupling	
Type	Coupling
Coupling object 1	
Name	CouplingAxis
Shuttle count	10
Cam automat reference	
Cam list reference	

When configuring a coupling object, it must be given a unique name. It will be used later in the application. The number of shuttles that can be coupled simultaneously via the coupling object is also configured. The number should be kept as low as possible since each superfluous setpoint generator consumes resources. Finally, a cam automat can be referenced and the cam functionality can be used.

Figure 49: Configuring a coupling object

10.3 Programming

When integrating the coupling into the application, it is recommended to program a step sequencer. The step sequencer described below is based on the one described in chapter 5.3 "Integration into the application" on page 26.

Sequence of steps

Each step listed in the step sequencer example is explained below. The steps that have not changed compared to the basic step sequencer are not described.

- Check trigger
- Get user data
- Send couple position
- Set couple object
- Couple shuttle
- Process
- Send next station
- Set user data

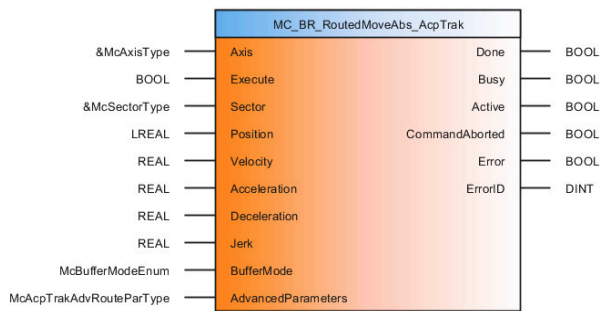


Figure 50: Function block MC_BR_RoutedMoveAbs_AcpTrak

Send couple position

In this step, the shuttle is sent to the position where it is then coupled to the axis.

For this, the function block MC_BR_RoutedMoveAbs_AcpTrak is used, which executes an absolute, elastic movement.

As soon as the movement is completed, the next step is jumped to.



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_RoutedMoveAbs_AcpTrak

Set couple object

In this step, the previously configured coupling object is added to the shuttle. The function block MC_BR_ShCouplingObjCmd_AcpTrak is used with the command mcACPTRAK_COUPLE_OBJ_SET.

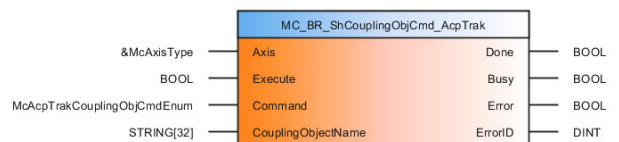


Figure 51: Function block MC_BR_ShCouplingObjCmd_AcpTrak

It must be taken into account that the shuttle is located in a user-defined sector.

Once the pairing object has been successfully added, the system jumps to the next step.



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks / MC_BR_ShCouplingObjCmd_AcpTrak

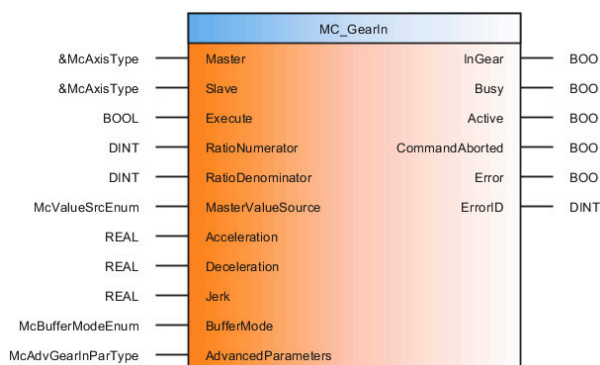


Figure 52: Function block MC_GearIn

Couple shuttle

In this step, the shuttle is coupled to the axis. The function block MC_GearIn from the "McAxis" library is used, which is also supported by ACOPOSTrak.

As soon as the output "InGear" shows that the coupling is active, the next step process is jumped to.



Drive technology / mapp Motion / Libraries / Core / McAxis / Function blocks / MC_GearIn

Process

In this step, the coupling between the shuttle and single axis is active. In this example, the single axis is controlled and moved via mapp Cockpit. As soon as the process has been successfully completed or aborted with a command, the system jumps to the next step.

Send next station

This step corresponds to the step process in the original step sequencer. Here, the shuttle is sent to the next process station with the function block MC_BR_RoutedMoveVel_AcpTrak.

10.4 Coupling between an axis and shuttle

In the following task, the master is a single axis controlled via mapp Cockpit.

Exercise: Coupling between an axis and shuttle

In this exercise, you will learn how to couple a shuttle to an axis using GearIn. The necessary steps are explained below.

- 1) Create a new process station.

Sector name	Beginning of sector			End of sector		
Sector_SealProduct	Seg_A_1	0.0	Start of segment	Seg_A_2	0.33	Start of segment

Table 4: Sector definition for process station Seal product

Process point name	Reference sector	Position	Position relative to
PP_SealProduct	Sector_SealProduct	0.1	Start of sector

Table 5: Process point definition for process station Seal product

- 2) Copy the process task "Process_GetProduct" and rename it to "Process_SealProduct".
- 3) Extend the local enumeration "Step_enum" according to the following table.

Old step name		New step name		Description
1	CHECK_TRIGGER	1	CHECK_TRIGGER	Read the event at the process point
2	GET_USERDATA	2	GET_USERDATA	Copy user data from the shuttle
-	-	3	SEND_COUPLE_POS	Absolute movement to the coupling position
-	-	4	SET_COUPLE_OBJ	Add slave shuttle coupling object
-	-	5	COUPLE_SHUTTLE	Enable coupling using GearIn
-	-	6	PROCESS	Control the single axis and wait for the "cmdS-topSealing" command.
3	PROCESS	7	SEND_NEXT_STATION	Movement to the next process station
4	SET_USERDATA	8	SET_USERDATA	Write processed user data back to the shuttle.

Table 6: Step definition for step sequencer

- 4) Declare new local variables and assign the values from the table in the INIT.

Variable name	Data type	Value	Description
CoupleObjName	String[25]	"CouplingAxis"	Name of the coupling object for shuttle/axis coupling from the "Assembly" feature
CouplingPosition	LREAL	0.545	Target position of the shuttle at the process sector for the start of the coupling

Table 7: Define the coupling object and coupling position

- 5) Single axis gSealAxis on mapp Motion - Add and configure basis

Name	Wert	Name	Wert
gSealAxis		gSealAxis	
Base type	Linear bounded	Base type	Linear bounded
Measurement unit	Millimeters	Movement limits	Internal
Measurement resolution	0.01	Position	
Count direction	Standard	Lower limit	-350
Movement limits	Internal	Upper limit	350
Position		Velocity	Basic
Velocity	Basic	Velocity	4000
Acceleration	Basic	Acceleration	Basic
Deceleration	Basic	Acceleration	40000
Alarms	None	Deceleration	Basic
		Deceleration	40000
		Alarms	None

Figure 53: gSealAxis configuration

- 6) Configure mapp Cockpit for single-axis control and enable the HMI application.
- 7) Add and reference an assembly feature configuration and create a new feature "CouplingAxis".
- 8) The process point is also used as a barrier with a ticket system, so that a single shuttle can only enter when the process station is empty.
- 9) Enhance the step sequencer with the newly declared steps and adapt the step descriptions.
- 10) For a description of the required function blocks, see the help documentation.
- 11) Use the command "cmdStopSealing" to resolve the coupling; the shuttle is then sent to the next process station "Process_ReleaseProduct" and colored orange (ShuttleUserdata.Color := Orange) in the Scene Viewer.
- 12) Test the application in the Scene Viewer via the Watch window and mapp Cockpit.



If the "McProfGen" module is missing during compilation, it must be changed via "Project / Runtime versions". Use the "Advanced" button to display the individual modules in the mapp Motion Technology Package and activate the "McProfGen" module with the version used.



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Function blocks
 Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Technical information / Coupling shuttles

10.5 Coupling between two shuttles

The procedure for a coupling between two shuttles with MC_GearIn is the same as for a coupling between a shuttle and an external axis.

Exercise: Coupling between two shuttles

In this exercise, you will learn how to couple two shuttles. This exercise is an extension of the previous exercise "Couple shuttle on axis". The structure of the coupling process is the same, except that a shuttle is specified as the master reference. Just like in the task, the master shuttle itself can be a slave of another coupling, e.g. with an external axis.

- 1) Declare a new local variable "CntShuttle" with the data type UINT.
- 2) Extend both coupling variables to an array with 2 elements each and assign the new values in INIT.

Variable name	Data type	Value	Description
CoupleObjName	String[25][1..2]	[1] 'CouplingAxis'	Names of the coupling objects for shuttle-axis coupling and shuttle-shuttle coupling.
		[2] 'CouplingShuttle'	
CouplingPosition	LREAL[1..2]	[1] 0.580	Target position of the shuttle at the process sector for the start of the coupling
		[2] 0.510	

- 3) The variable "ShuttleAxis" must also be extended to an array with 2 elements in order to store both shuttle references.
- 4) Add an additional "CouplingShuttles" coupling feature to the assembly feature configuration.
- 5) The sequence of the step sequencer must be adapted so that the two shuttles are sent to the coupling position one after the other.
- 6) After reaching this position, the coupling objects are added to them one after the other and the coupling is enabled.
- 7) When the coupling is terminated with the command "cmdStopSealing", the two shuttles are sent one after the other with 1.5 m/s to the next process station and the changed user data (ShuttleUserdata.Color : = Orange) is transferred.
- 8) Test the application in the Scene Viewer via the Watch window and mapp Cockpit.



Drive technology / mapp Motion / Libraries / Core / McAcpTrak / Technical information / Coupling shuttles

11 Interlinking

The modular design and process-oriented software concept of ACOPOSTrak allow it to be connected to conventional conveyor belts.

11.1 Concept

Interlinking between ACOPOSTrak and conventional conveyor belts is possible both vertically and horizontally. The basic concept is that only certain parts and process areas are implemented with ACOPOSTrak. As soon as a shuttle is transported into the assembly workspace, it must be added at the correct position via the application in order to be able to control it. Once the shuttle has been successfully added, it can be controlled and the process can be started. Once the process is finished, the shuttle must be deleted from the assembly before it is permitted to leave it; otherwise, an error will occur.



Figure 54: Interlinking at a bottling line

11.2 Implementation

The shuttle must be manually added to the system via the application just like menu option "Concept". The function block MC_BR_SecAddShuttle_AcpTrak should be used to do this. This function block adds the shuttle to a reference sector at the specified position. If the shuttle is added during a movement, an initial velocity can be specified.

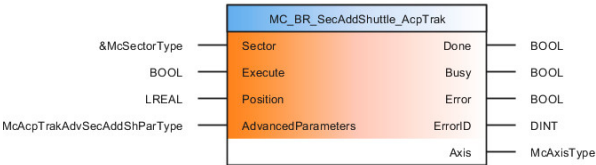


Figure 55: Function block MC_BR_SecAddShuttle_AcpTrak

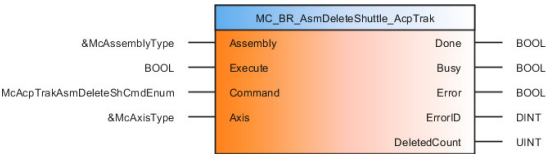


Figure 56: Function block MC_BR_AsmDeleteShuttle_AcpTrak

Before a shuttle should exit the track again, it must first be deleted via the application. The function block MC_BR_AsmDeletShuttle_AcpTrak is required for this. After the shuttle is deleted, it is necessary to ensure that the physical shuttle actually exits the track; otherwise, a collision may occur due to the lack of collision monitoring.

11.3 Accuracy

Since adding a shuttle activates the controllers at this position, the physical shuttle must also be at this position. There is a maximum tolerance of ± 2 mm between the physical and logical position. If the difference is too large, this can lead to an unintentional malfunction of the system (e.g. uncontrolled movement of the shuttle).

The graphic shows the logical position (black) with the tolerance range (red) on the sector. The physical position of the shuttle (green) is slightly offset, but still within the tolerance range of ± 2 mm.

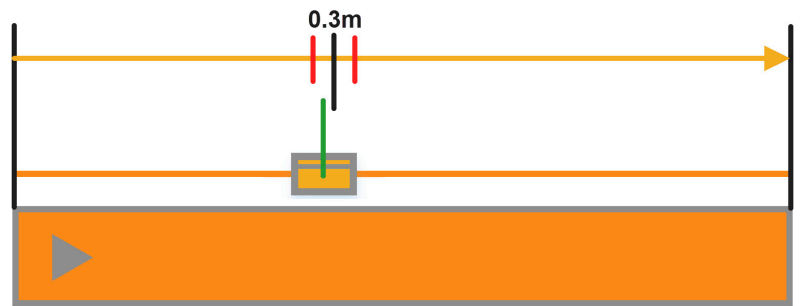


Figure 57: Shuttle position tolerance range

12 Summary

ACOPOStrak is the flexible track system for ultimate production efficiency.

The core of the track system is a linear motor assembled from four types of modular segments: A straight segment, a 45° segment and two 22.5° segments – one curved to the right, the other to the left. With its modular, flexible system concept, ACOPOStrak enables completely new machine designs.

All the steps required for getting ACOPOStrak up and running quickly are completed directly in Automation Studio. The extensive range of software functions, for development and dimensioning as well as configuration, programming and commissioning, reduce the time needed for the project considerably.

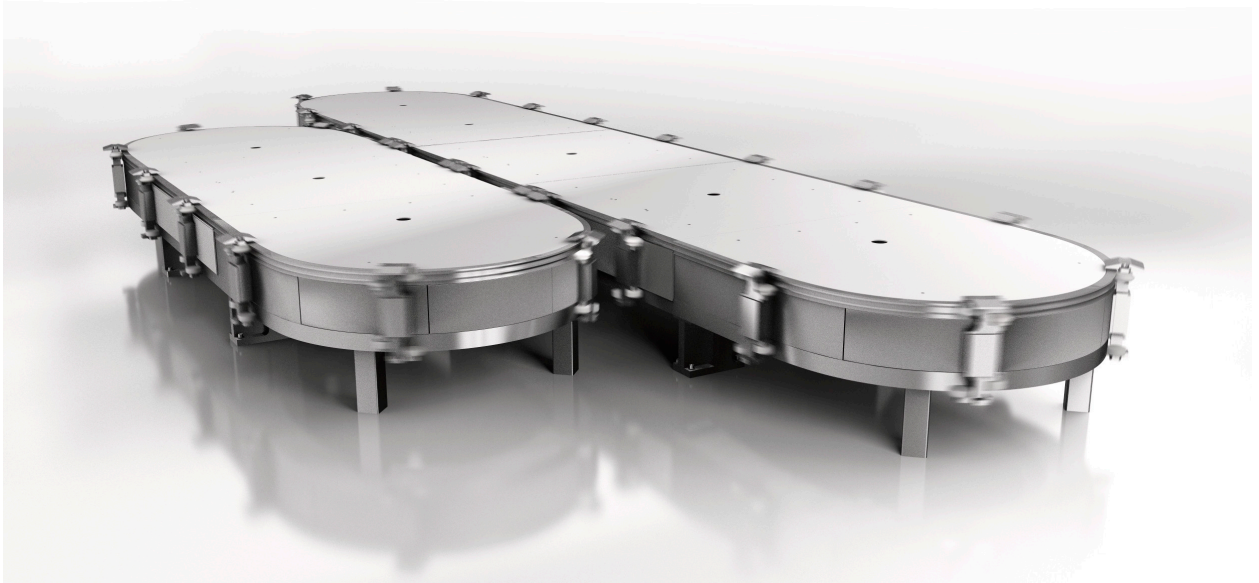


Figure 58: ACOPOStrak

Developers also benefit from process-oriented programming. The programmer describes rules that define the product flow on the track. This is faster than individually programming a large number of axes or shuttles and maximizes the flexibility of the system. Autonomous traffic control with integrated collision avoidance takes further work off the hands of developers.

The product flow at the track is controlled by so-called process points. At process points, shuttle-specific user data can be read and written to again; these determine the further course of the shuttle. Information about a product can be stored directly on the shuttle, and products with poor quality can be removed from the process using the high-speed diverter.

The intelligent system software of ACOPOStrak can be easily configured and programmed in Automation Studio and includes many highlights, such as different types of movements, the ability to use process points as barriers and the ability to couple shuttles with external axes or other shuttles, just to name a few.

In addition, the modular design and process-oriented software concept of ACOPOStrak allow it to be combined with conventional conveyor belts. Implementation with ACOPOStrak can thus be limited to certain plant parts and process areas, which results in more flexibility and cost efficiency.

13 Solutions and appendix

13.1 Solution: POWERLINK - Distribution

Here you will find the solution to the exercise "POWERLINK - Distribution" on page 9:

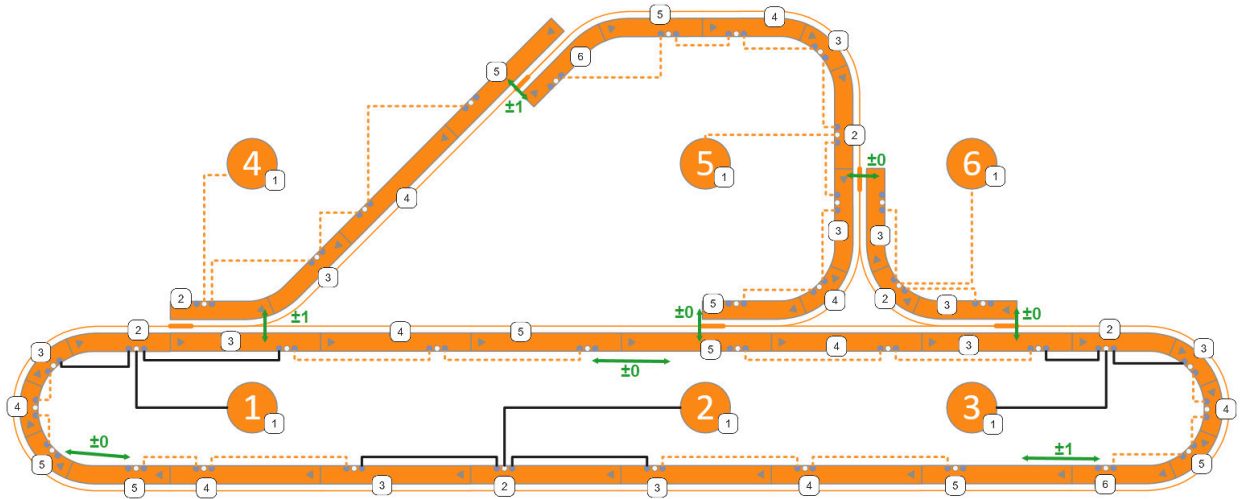


Figure 59: POWERLINK dimensioning solution

13.2 Solution: Electrical dimensioning

Here you will find the configuration for electrical dimensioning in the INIT of the dimensioning task of the exercise "Electrical dimensioning" on page 23.

```
// configuration

// general parameter
TrakDesign.AssemblyRef      := ADR(gAssembly_1);
TrakDesign.AssemblyName    := 'gAssembly_1';
TrakDesign.EnableCalc      := TRUE;
TrakDesign.CmdReset        := FALSE;
TrakDesign.CmdSaveReport   := FALSE;
TrakDesign.CmdRecordDiag   := FALSE;

TrakDesign.ParaShuttleType := 100;
TrakDesign.ParaTempAmbient := 25;

// TrakDesign.ParaMechShuttleIx := 1;
TrakDesign.ParaFileDevice    := 'User';
TrakDesign.ParaFileNameReport := 'TrakDesignReport.csv';
TrakDesign.ParaFileNameDiag  := 'TrakDesignDiag.dat';
TrakDesign.ParaFileCsvStyle  := 0;
TrakDesign.ParaDiagPrescale  := 10;

// shuttle payload parameter
// carrier
TrakDesign.Carrier.ParaMass      := 0.0; // [kg]
TrakDesign.Carrier.ParaMassCenterX := 0.0; // [m]
TrakDesign.Carrier.ParaMassCenterY := 0.0; // [m]
TrakDesign.Carrier.ParaMassCenterZ := 0.0; // [m]
TrakDesign.Carrier.ParaMassInertiaZZ := 0.0; // [kg m^2]
// product
TrakDesign.Payload[1].ParaName := 'no product on carrier';
TrakDesign.Payload[1].ParaMass := 0.0; // [kg]
TrakDesign.Payload[1].ParaMassCenterX := 0.0; // [m]
```



```

TrakDesign.Payload[1].ParaMassCenterY      := 0.0;      // [m]
TrakDesign.Payload[1].ParaMassCenterZ      := 0.0;      // [m]
TrakDesign.Payload[1].ParaMassInertiaZZ    := 0.0;      // [kg m^2]
// external force
TrakDesign.Payload[1].ParaExtForceX        := 0.0;      // [N]
TrakDesign.Payload[1].ParaExtForceY        := 0.0;      // [N]
TrakDesign.Payload[1].ParaExtForceZ        := 0.0;      // [N]
TrakDesign.Payload[1].ParaExtForceAppPointX := 0.0;      // [m]
TrakDesign.Payload[1].ParaExtForceAppPointY := 0.0;      // [m]
TrakDesign.Payload[1].ParaExtForceAppPointZ := 0.0;      // [m]

FOR i:=1 TO TRAK_MAX_SHUTTLE DO
    TrakDesign.Shuttle[i].ParaPayloadIx      := 1;
    // TrakDesign.Shuttle[i].ParaAutoPayload  := 0;
    // TrakDesign.Shuttle[i].ParaInitialOrientationFlip := 1;
END_FOR

// electrical segment assembly
TrakDesign.Segment[1].ParaName      := 'Seg_A_1';
TrakDesign.Segment[1].ParaType      := TYPE_AA;
TrakDesign.Segment[1].ParaIxCable   := 2;

TrakDesign.Segment[2].ParaName      := 'Seg_A_2';
TrakDesign.Segment[2].ParaType      := TYPE_AA;
TrakDesign.Segment[2].ParaIxCable   := 2;

TrakDesign.Segment[3].ParaName      := 'Seg_A_3';
TrakDesign.Segment[3].ParaType      := TYPE_AA;
TrakDesign.Segment[3].ParaIxCable   := 2;

TrakDesign.Segment[4].ParaName      := 'Seg_A_4';
TrakDesign.Segment[4].ParaType      := TYPE_AB;
TrakDesign.Segment[4].ParaIxCable   := 2;

TrakDesign.Segment[5].ParaName      := 'Seg_A_5';
TrakDesign.Segment[5].ParaType      := TYPE_BB;
TrakDesign.Segment[5].ParaIxCable   := 2;

TrakDesign.Segment[6].ParaName      := 'Seg_A_6';
TrakDesign.Segment[6].ParaType      := TYPE_BB;
TrakDesign.Segment[6].ParaIxCable   := 3;

TrakDesign.Segment[7].ParaName      := 'Seg_A_7';
TrakDesign.Segment[7].ParaType      := TYPE_BB;
TrakDesign.Segment[7].ParaIxCable   := 3;

TrakDesign.Segment[8].ParaName      := 'Seg_A_8';
TrakDesign.Segment[8].ParaType      := TYPE_BA;
TrakDesign.Segment[8].ParaIxCable   := 3;

TrakDesign.Segment[9].ParaName      := 'Seg_A_9';
TrakDesign.Segment[9].ParaType      := TYPE_AA;
TrakDesign.Segment[9].ParaIxCable   := 3;

TrakDesign.Segment[10].ParaName     := 'Seg_A_10';
TrakDesign.Segment[10].ParaType     := TYPE_AA;
TrakDesign.Segment[10].ParaIxCable  := 3;

TrakDesign.Segment[11].ParaName     := 'Seg_A_11';
TrakDesign.Segment[11].ParaType     := TYPE_AA;
TrakDesign.Segment[11].ParaIxCable  := 1;

TrakDesign.Segment[12].ParaName     := 'Seg_A_12';
TrakDesign.Segment[12].ParaType     := TYPE_AB;
TrakDesign.Segment[12].ParaIxCable  := 1;

```

```

TrakDesign.Segment[13].ParaName      := 'Seg_A_13';
TrakDesign.Segment[13].ParaType      := TYPE_BB;
TrakDesign.Segment[13].ParaIxCable  := 1;

TrakDesign.Segment[14].ParaName      := 'Seg_A_14';
TrakDesign.Segment[14].ParaType      := TYPE_BB;
TrakDesign.Segment[14].ParaIxCable  := 1;

TrakDesign.Segment[15].ParaName      := 'Seg_A_15';
TrakDesign.Segment[15].ParaType      := TYPE_BB;
TrakDesign.Segment[15].ParaIxCable  := 1;

TrakDesign.Segment[16].ParaName      := 'Seg_A_16';
TrakDesign.Segment[16].ParaType      := TYPE_BA;
TrakDesign.Segment[16].ParaIxCable  := 2;

// electrical power cable assembly
TrakDesign.Cable[1].ParaLength       := 10.0;
TrakDesign.Cable[1].ParaIxSupply     := 1;

TrakDesign.Cable[2].ParaLength       := 10.0;
TrakDesign.Cable[2].ParaIxSupply     := 1;

TrakDesign.Cable[3].ParaLength       := 10.0;
TrakDesign.Cable[3].ParaIxSupply     := 1;

// electrical supply assembly
TrakDesign.Supply[1].ParaType         := PS080;
TrakDesign.Supply[1].ParaIxSupplyParallel := 2;

TrakDesign.Supply[2].ParaType         := PS080;
TrakDesign.Supply[2].ParaIxSupplyParallel := 0;

```

13.3 Solution: Accessing the user data

Here you will find the program code for the task "Process_Test" from the exercise "Accessing the user data" on page 26:
Data type declaration:

```

TYPE
    Step_enum :
    (
        CHECK_TRIGGER,
        GET_USERDATA,
        PROCESS,
        SET_USERDATA
    );
END_TYPE

```

Variable declaration:

```

VAR
    Step : Step_enum;
    MC_BR_TriggerPointEnable_AcpTrak_0 : MC_BR_TriggerPointEnable_AcpTrak;
    CmdEnableProcess : BOOL;
    MC_BR_TriggerPointGetInfo_AcpTrak_0 : MC_BR_TriggerPointGetInfo_AcpTrak;
    ShuttleAxis : McAxisType;
    MC_BR_ShCopyUserData_AcpTrak_0 : MC_BR_ShCopyUserData_AcpTrak;
    ShuttleUserData : ShuttleUserData_typ;
END_VAR

```

Program code:

```

PROGRAM _INIT
    (* Local variable to enable/disable the the process *)
    CmdEnableProcess := TRUE;

    (* Processpointreference for Functionblocks *)

```

```

MC_BR_TrqPointEnable_AcpTrak_0.ProcessPoint := ADR(PP_Test);
MC_BR_TrqPointGetInfo_AcpTrak_0.ProcessPoint := ADR(PP_Test);

END_PROGRAM

PROGRAM _CYCLIC

CASE Step OF
CHECK_TRIGGER:
    (* When the process is enabled and the assembly is ready,
    the process will be activated *)
    IF ((CmdEnableProcess) AND (gTrakAsm.Status.Ready)) THEN
        MC_BR_TrqPointEnable_AcpTrak_0.Enable := TRUE;

        (* If a shuttle passes the triggerpoint then
        get the axis reference *)
        IF ((MC_BR_TrqPointEnable_AcpTrak_0.Valid) AND
            (MC_BR_TrqPointEnable_AcpTrak_0.EventCount > 0)) THEN
            MC_BR_TrqPointGetInfo_AcpTrak_0.Execute := TRUE;
        END_IF
    ELSE (* Else the processpoint will be disabled *)
        MC_BR_TrqPointEnable_AcpTrak_0.Enable := FALSE;
    END_IF

    (* If axis reference is available,
    save it and go to the next step *)
    IF EDGEPOS (MC_BR_TrqPointGetInfo_AcpTrak_0.Done) THEN
        (* Save axis reference in local variable for use
        in step machine *)
        ShuttleAxis := MC_BR_TrqPointGetInfo_AcpTrak_0.TrqPointInfo.Axis;

        MC_BR_TrqPointGetInfo_AcpTrak_0.Execute := FALSE;
        Step := GET_USERDATA;
    END_IF

GET_USERDATA:
    (* The userdata is copied on a local variable *)
    MC_BR_ShCopyUserData_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCopyUserData_AcpTrak_0.DataAddress := ADR(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.DataSize := SIZEOF(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.Mode := mcACPTRAK_USERDATA_GET;

    (* After copying go to the next step *)
    IF EDGEPOS (MC_BR_ShCopyUserData_AcpTrak_0.Done) THEN
        MC_BR_ShCopyUserData_AcpTrak_0.Execute := FALSE;
        Step := PROCESS;
    END_IF

PROCESS:
    (* The shuttle color is changed according to the actual color*)
    IF (ShuttleUserData.Color = GREEN) THEN
        (* All green shuttles will turn red *)
        ShuttleUserData.Color := RED;
    ELSE
        (* The rest of the shuttles will turn green *)
        ShuttleUserData.Color := GREEN;
    END_IF

    Step := SET_USERDATA;

SET_USERDATA:
    (* The modified userdata will be copied back on the shuttle *)
    MC_BR_ShCopyUserData_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := TRUE;

```

```
MC_BR_ShCopyUserData_AcpTrak_0.DataAddress := ADR(ShuttleUserData);
MC_BR_ShCopyUserData_AcpTrak_0.DataSize := SIZEOF(ShuttleUserData);
MC_BR_ShCopyUserData_AcpTrak_0.Mode := mcACPTRAK_USERDATA_SET;

(* After copying go to the first step and wait for new event *)
IF EDGEPOS(MC_BR_ShCopyUserData_AcpTrak_0.Done) THEN
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := FALSE;
    Step := CHECK_TRIGGER;
END_IF

END_CASE

(* Call all function blocks *)
MC_BR_TrjPointEnable_AcpTrak_0();
MC_BR_TrjPointGetInfo_AcpTrak_0();
MC_BR_ShCopyUserData_AcpTrak_0();

END_PROGRAM

PROGRAM _EXIT

END_PROGRAM
```

13.4 Basic step sequencer for process stations

The following figure shows the step sequencer as a flowchart, which can be used as a basis for each process station.

The step sequencer consists of 4 steps, which are described in more detail in chapter 5.3 "Integration into the application" on page 26.

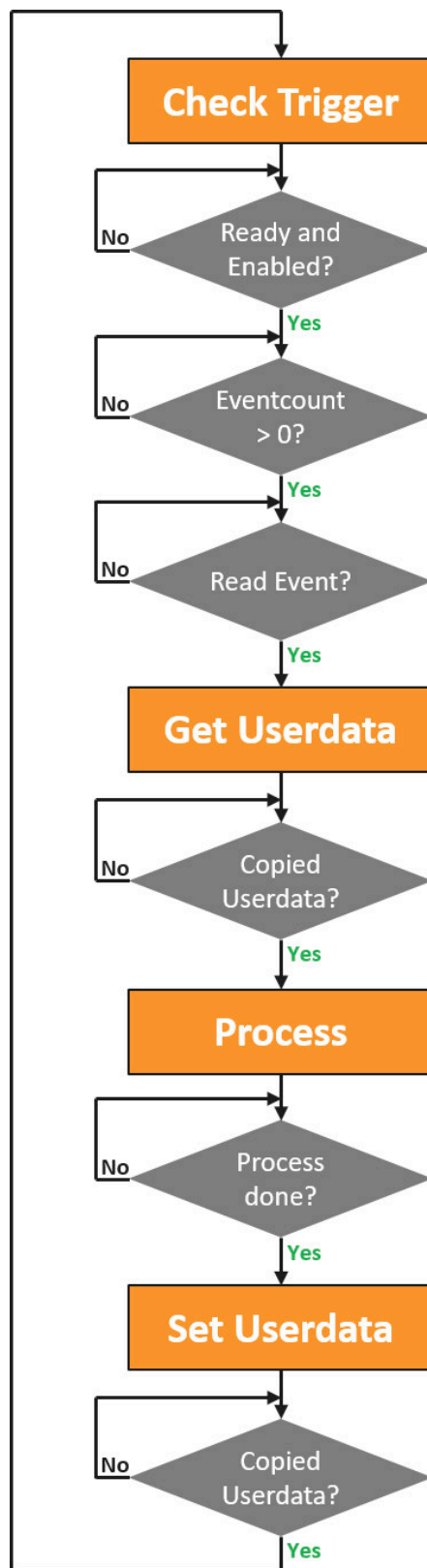


Figure 60: Step sequencer represented as a flowchart

13.5 Solution: Developing a process station

Here you will find the program code for the step "Process" in the task "Process_GetProduct" from the exercise "Developing a process station" on page 28:

The structure of the task "Process_ReleaseProduct" is identical. Only the parameters need to be adjusted for each process station.

PROCESS:

```
(* Change Shuttlecolor according to exercise *)
ShuttleUserData.Color := GREEN;

(* Send the shuttle to the next process station *)
MC_BR_RoutedMoveVel_AcpTrak_0.Axis := ADR(ShuttleAxis);
MC_BR_RoutedMoveVel_AcpTrak_0.Execute := TRUE;
MC_BR_RoutedMoveVel_AcpTrak_0.Sector := ADR(Sector_ReleaseProduct);
MC_BR_RoutedMoveVel_AcpTrak_0.Position := 0.0;
MC_BR_RoutedMoveVel_AcpTrak_0.Velocity := 1.0;
MC_BR_RoutedMoveVel_AcpTrak_0.RouteVelocity := 1.0;
MC_BR_RoutedMoveVel_AcpTrak_0.Acceleration := 20.0;
MC_BR_RoutedMoveVel_AcpTrak_0.Deceleration := 20.0;
MC_BR_RoutedMoveVel_AcpTrak_0.Jerk := 0.0;
MC_BR_RoutedMoveVel_AcpTrak_0.BufferMode := mcABORTING;
MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.StartDirection
    := mcDIR_UNDEFINED;
MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.EndDirection
    := mcDIR_POSITIVE;
MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.ShuttleOrientation
    := mcDIR_UNDEFINED;

(* When the movement is active, reset the execute *)
IF EDGEPOS(MC_BR_RoutedMoveVel_AcpTrak_0.Active) THEN
    MC_BR_RoutedMoveVel_AcpTrak_0.Execute := FALSE;
    Step := SET_USERDATA;
END_IF
```

13.6 Solution: Extending the assembly with diverters

Solution for exercise "Extension of the assembly with diverters" on page 30:

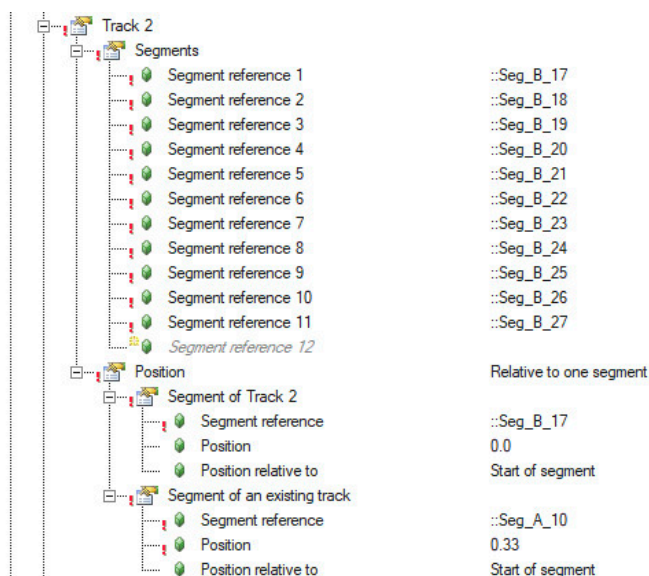


Figure 61: Solution for the assembly configuration of the second track

13.7 Solution: Process station with rigid movement

Here you will find the program code for the task "Process_FillProduct" from the exercise "[Process station with rigid movement](#)" on page 32:

The code in the "FillEnd" action has the same structure as the one in the "FillStart" action. Only in the "Process" step is the function block MC_BR_RoutedMoveVelocity_AcpTrak used instead of the function block MC_MoveVelocity.

Program code - Init

```
PROGRAM _INIT
    (* Local variable to enable/disable the the process *)
    CmdEnableProcess := TRUE;

    (* Processpointreference for Functionblocks *)
    MC_BR_TriggerPointEnable_FillStart.ProcessPoint := ADR(PP_FillProductStart);
    MC_BR_TriggerPointGetInfo_FillStart.ProcessPoint := ADR(PP_FillProductStart);

    MC_BR_TriggerPointEnable_FillEnd.ProcessPoint := ADR(PP_FillProductEnd);
    MC_BR_TriggerPointGetInfo_FillEnd.ProcessPoint := ADR(PP_FillProductEnd);
END_PROGRAM
```

Program code - Cyclic

```
PROGRAM _CYCLIC
    (* Call all actions *)
    FillStart;
    FillEnd;

    (* Call all function blocks *)
    MC_BR_TriggerPointEnable_FillStart();
    MC_BR_TriggerPointGetInfo_FillStart();
    MC_BR_ShCopyUserData_FillStart();
    MC_MoveVelocity_FillStart();

    MC_BR_TriggerPointEnable_FillEnd();
    MC_BR_TriggerPointGetInfo_FillEnd();
    MC_BR_ShCopyUserData_FillEnd();
    MC_BR_RoutedMoveVel_FillEnd();
END_PROGRAM
```

Program code - Action "FillStart"

```
ACTION FillStart:
    CASE Step_FillStart OF
        CHECK_TRIGGER:
            (* When the process is enabled and the assembly is ready,
            the process will be activated *)
            IF ((CmdEnableProcess) AND (gTrakAsm.Status.Ready)) THEN
                MC_BR_TriggerPointEnable_FillStart.Enable := TRUE;

                (* If a shuttle passes the triggerpoint then
                get the axis reference *)
                IF (MC_BR_TriggerPointEnable_FillStart.Valid)
                AND (MC_BR_TriggerPointEnable_FillStart.EventCount > 0) THEN
                    MC_BR_TriggerPointGetInfo_FillStart.Execute := TRUE;
                END_IF
            ELSE
                MC_BR_TriggerPointEnable_FillStart.Enable := FALSE;
            END_IF

            (* If axis reference is available,
            save it and go to the next step *)
            IF EDGEPOS(MC_BR_TriggerPointGetInfo_FillStart.Done) THEN
                (* Save axis reference in local variable for use
                in step machine *)
                ShuttleAxis_FillStart :=
                    MC_BR_TriggerPointGetInfo_FillStart.TriggerPointInfo.Axis;
```

```

        MC_BR_TrqPointGetInfo_FillStart.Execute := FALSE;
        Step_FillStart := GET_USERDATA;
    END_IF

GET_USERDATA:
    (* The userdata is copied on a local variable *)
    MC_BR_ShCopyUserData_FillStart.Axis := ADR(ShuttleAxis_FillStart);
    MC_BR_ShCopyUserData_FillStart.Execute := TRUE;
    MC_BR_ShCopyUserData_FillStart.DataAddress :=
        ADR(ShuttleUserData_FillStart);
    MC_BR_ShCopyUserData_FillStart.DataSize :=
        SIZEOF(ShuttleUserData_FillStart);
    MC_BR_ShCopyUserData_FillStart.Mode := mcACPTRAK_USERDATA_GET;

    (* After copying go to the next step *)
    IF EDGEPOS(MC_BR_ShCopyUserData_FillStart.Done) THEN
        MC_BR_ShCopyUserData_FillStart.Execute := FALSE;
        Step_FillStart := PROCESS;
    END_IF

PROCESS:
    (* Change Shuttlecolor according to exersice *)
    ShuttleUserData_FillStart.Color := ORANGE;

    (* Send the shuttle with rigid movement along the sector *)
    MC_MoveVelocity_FillStart.Axis := ADR(ShuttleAxis_FillStart);
    MC_MoveVelocity_FillStart.Execute := TRUE;
    MC_MoveVelocity_FillStart.Direction := mcDIR_POSITIVE;
    MC_MoveVelocity_FillStart.Velocity := 0.5;
    MC_MoveVelocity_FillStart.Acceleration := 20.0;
    MC_MoveVelocity_FillStart.Deceleration := 20.0;
    MC_MoveVelocity_FillStart.Jerk := 0.0;
    MC_MoveVelocity_FillStart.BufferMode := mcABORTING;

    (* When the movement is active, reset the execute *)
    IF EDGEPOS(MC_MoveVelocity_FillStart.Active) THEN
        MC_MoveVelocity_FillStart.Execute := FALSE;
        Step_FillStart := SET_USERDATA;
    END_IF

SET_USERDATA:
    (* The modified userdata will be copied back on the shuttle *)
    MC_BR_ShCopyUserData_FillStart.Axis := ADR(ShuttleAxis_FillStart);
    MC_BR_ShCopyUserData_FillStart.Execute := TRUE;
    MC_BR_ShCopyUserData_FillStart.DataAddress :=
        ADR(ShuttleUserData_FillStart);
    MC_BR_ShCopyUserData_FillStart.DataSize :=
        SIZEOF(ShuttleUserData_FillStart);
    MC_BR_ShCopyUserData_FillStart.Mode := mcACPTRAK_USERDATA_SET;

    (* After copying go to the first step and wait for new event *)
    IF EDGEPOS(MC_BR_ShCopyUserData_FillStart.Done) THEN
        MC_BR_ShCopyUserData_FillStart.Execute := FALSE;
        Step_FillStart := CHECK_TRIGGER;
    END_IF

END_CASE
END_ACTION

```

Solution: Process point as barrier

Here you will find the additional program code for the task "Process_FillProduct" from the exercise "Process point as barrier" on page 34. The code is added to the existing program of the exercise "Process station with rigid movement" on page 32.


```

PROGRAM _INIT
    (* Command for barrier to add tickets *)
    cmdFillStart := TRUE;

    (* Barrier parameters *)
    MC_BR_BarrierCommand_FillStart.ProcessPoint := ADR(PP_FillProductStart);
    MC_BR_BarrierCommand_FillStart.Command := mcACPTRAK_BARRIER_ADD_TICKETS;
    MC_BR_BarrierCommand_FillStart.AdvancedParameters.TicketCount := 2;
END_PROGRAM

PROGRAM _CYCLIC
    (* Waiting for a command *)
    IF (cmdFillStart) THEN
        cmdFillStart := FALSE;
        MC_BR_BarrierCommand_FillStart.Execute := TRUE;
    END_IF

    (* When the tickets are added, reset the execute *)
    IF EDGEPOS(MC_BR_BarrierCommand_FillStart.Done) THEN
        MC_BR_BarrierCommand_FillStart.Execute := FALSE;
    END_IF

    (* Call the function block *)
    MC_BR_BarrierCommand_FillStart();
END_PROGRAM

```

13.8 Solution: Coupling between an axis and shuttle

Here you will find the program code for the task "Process_SealProduct" from the exercise "[Coupling between an axis and shuttle](#)" on page 37:

```

PROGRAM _INIT
    (* Local variable to enable/disable the the process *)
    CmdEnableProcess := TRUE;

    (* Processpointreference for Functionblocks *)
    MC_BR_TriggerPointEnable_AcpTrak_0.ProcessPoint := ADR(PP_SealProduct);
    MC_BR_TriggerPointGetInfo_AcpTrak_0.ProcessPoint := ADR(PP_SealProduct);

    (* Barrier configuration *)
    MC_BR_BarrierCommand_AcpTrak_0.ProcessPoint := ADR(PP_SealProduct);
    MC_BR_BarrierCommand_AcpTrak_0.Command := mcACPTRAK_BARRIER_ADD_TICKETS;
    MC_BR_BarrierCommand_AcpTrak_0.AdvancedParameters.TicketCount := 1;
    MC_BR_BarrierCommand_AcpTrak_0.Execute := TRUE; (* To initialize the barrier *)

    (* Coupling object *)
    CoupleObjName := 'CouplingAxis';

    (* Coupling position *)
    CouplingPosition := 0.545;
END_PROGRAM

PROGRAM _CYCLIC
    CASE Step OF
        CHECK_TRIGGER:
            (* When the process is enabled and the assembly is ready,
            the process will be activated *)
            IF ((CmdEnableProcess) AND (gTrakAsm.Status.Ready)) THEN
                MC_BR_TriggerPointEnable_AcpTrak_0.Enable := TRUE;

                (* If a shuttle passes the triggerpoint then get
                the axis reference *)
                IF (MC_BR_TriggerPointEnable_AcpTrak_0.Valid) AND

```

```

        (MC_BR_TrjPointEnable_AcpTrak_0.EventCount > 0)) THEN
        MC_BR_TrjPointGetInfo_AcpTrak_0.Execute := TRUE;
    END_IF
ELSE (* Else the processpoint will be disabled *)
    MC_BR_TrjPointEnable_AcpTrak_0.Enable := FALSE;
END_IF

(* If axis reference is available,
   save it and go to the next step *)
IF EDGEPOS(MC_BR_TrjPointGetInfo_AcpTrak_0.Done) THEN
    (* Save axis reference in local variable
       for use in step machine *)
    ShuttleAxis := MC_BR_TrjPointGetInfo_AcpTrak_0.TrjPointInfo.Axis;

    MC_BR_TrjPointGetInfo_AcpTrak_0.Execute := FALSE;
    Step := GET_USERDATA;
END_IF

GET_USERDATA:
    (* The userdata is copied on a local variable *)
    MC_BR_ShCopyUserData_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCopyUserData_AcpTrak_0.DataAddress := ADR(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.DataSize := SIZEOF(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.Mode := mcACPTRAK_USERDATA_GET;

    (* After copying go to the next step *)
    IF EDGEPOS(MC_BR_ShCopyUserData_AcpTrak_0.Done) THEN
        MC_BR_ShCopyUserData_AcpTrak_0.Execute := FALSE;
        Step := SEND_COUPLE_POS;
    END_IF

SEND_COUPLE_POS:
    (* Send the shuttle to the target position for coupling *)
    MC_BR_RoutedMoveAbs_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_RoutedMoveAbs_AcpTrak_0.Execute := TRUE;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Sector := ADR(Sector_SealProduct);
    MC_BR_RoutedMoveAbs_AcpTrak_0.Position := CouplingPosition;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Velocity := 1.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Acceleration := 20.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Deceleration := 20.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Jerk := 0.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.BufferMode := mcABORTING;
    MC_BR_RoutedMoveAbs_AcpTrak_0.AdvancedParameters.StartDirection
        := mcDIR_UNDEFINED;
    MC_BR_RoutedMoveAbs_AcpTrak_0.AdvancedParameters.EndDirection
        := mcDIR_POSITIVE;
    MC_BR_RoutedMoveAbs_AcpTrak_0.AdvancedParameters.ShuttleOrientation
        := mcDIR_UNDEFINED;

    (* When the movement is finished, go to the next step
       and let in the next shuttle*)
    IF EDGEPOS(MC_BR_RoutedMoveAbs_AcpTrak_0.Done) THEN
        MC_BR_RoutedMoveAbs_AcpTrak_0.Execute := FALSE;
        Step := SET_COUPLE_OBJ;
    END_IF

SET_COUPLE_OBJ:
    (* Place the coupling object on the slave (shuttle) *)
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Command := mcACPTRAK_COUPLE_OBJ_SET;
    MC_BR_ShCouplingObjCmd_AcpTrak_0.CouplingObjectName := CoupleObjName;

    (* When the function block is finished, go to the next step *)

```

```

IF EDGEPOS (MC_BR_ShCouplingObjCmd_AcpTrak_0.Done) THEN
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Execute := FALSE;
    Step := COUPLE_SHUTTLE;
END_IF

COUPLE_SHUTTLE:
    (* Couple the shuttle to the axis *)
    MC_GearIn_0.Master := ADR(gSealAxis);
    MC_GearIn_0.Slave := ADR(ShuttleAxis);
    MC_GearIn_0.Execute := TRUE;
    MC_GearIn_0.Acceleration := 40;
    MC_GearIn_0.Deceleration := 40;
    MC_GearIn_0.BufferMode := mcABORTING;
    MC_GearIn_0.RatioNumerator := 1000;
    MC_GearIn_0.RatioDenominator := 1000;
    MC_GearIn_0.MasterValueSource := mcVALUE_ACTUAL;
    MC_GearIn_0.AdvancedParameters.MasterMaxVelocity := 4; (* 4m/s *)

    (* When the coupling is active, go to the next step *)
    IF EDGEPOS (MC_GearIn_0.InGear) THEN
        MC_GearIn_0.Execute := FALSE;
        Step := PROCESS;
    END_IF

PROCESS:
    (* Move the axis via mapp Cockpit *)

    (* Wait for stop command *)
    IF (cmdStopSealing) THEN
        cmdStopSealing := FALSE;
        Step := SEND_NEXT_STATION;
    END_IF

SEND_NEXT_STATION:
    (* Change the color of the Shuttle *)
    ShuttleUserData.Color := ORANGE;

    MC_BR_RoutedMoveVel_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_RoutedMoveVel_AcpTrak_0.Execute := TRUE;
    MC_BR_RoutedMoveVel_AcpTrak_0.Sector := ADR(Sector_ReleaseProduct);
    MC_BR_RoutedMoveVel_AcpTrak_0.Position := 0.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.Velocity := 1.5;
    MC_BR_RoutedMoveVel_AcpTrak_0.RouteVelocity := 1.5;
    MC_BR_RoutedMoveVel_AcpTrak_0.Acceleration := 20.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.Deceleration := 20.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.Jerk := 0.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.BufferMode := mcABORTING;
    MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.StartDirection
        := mcDIR_UNDEFINED;
    MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.EndDirection
        := mcDIR_POSITIVE;
    MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.ShuttleOrientation
        := mcDIR_UNDEFINED;

    (* When the movement of the shuttle is active, go to the next step *)
    IF EDGEPOS (MC_BR_RoutedMoveVel_AcpTrak_0.Active) THEN
        MC_BR_RoutedMoveVel_AcpTrak_0.Execute := FALSE;
        Step := SET_USERDATA;
    END_IF

SET_USERDATA:
    (* The modified userdata will be copied back on the shuttle *)
    MC_BR_ShCopyUserData_AcpTrak_0.Axis := ADR(ShuttleAxis);
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCopyUserData_AcpTrak_0.DataAddress := ADR(ShuttleUserData);

```

```

MC_BR_ShCopyUserData_AcpTrak_0.DataSize := SIZEOF(ShuttleUserData);
MC_BR_ShCopyUserData_AcpTrak_0.Mode := mcACPTRAK_USERDATA_SET;

(* After copying go to the first step and wait for a new event
and let the next shuttle into the processstation*)
IF EDGEPOS(MC_BR_ShCopyUserData_AcpTrak_0.Done) THEN
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := FALSE;

    MC_BR_BarrierCommand_AcpTrak_0.Execute := TRUE;
    Step := CHECK_TRIGGER;
END_IF

END_CASE

(* Call all function blocks *)
MC_BR_TriggerPointEnable_AcpTrak_0();
MC_BR_TriggerPointGetInfo_AcpTrak_0();
MC_BR_ShCopyUserData_AcpTrak_0();
MC_BR_RoutedMoveAbs_AcpTrak_0();
MC_BR_ShCouplingObjCmd_AcpTrak_0();
MC_GearIn_0();
MC_BR_RoutedMoveVel_AcpTrak_0();

MC_BR_BarrierCommand_AcpTrak_0();
(* When adding tickets is done, reset the execute *)
IF EDGEPOS(MC_BR_BarrierCommand_AcpTrak_0.Done) THEN
    MC_BR_BarrierCommand_AcpTrak_0.Execute := FALSE;
END_IF

END_PROGRAM

PROGRAM _EXIT

END_PROGRAM

```

13.9 Solution: Coupling between two shuttles

Here you will find the program code for the task "Process_SealProduct" from the exercise "Coupling between two shuttles" on page 38:

```

PROGRAM _INIT
    (* Local variable to enable/disable the the process *)
    CmdEnableProcess := TRUE;

    (* Processpointreference for Functionblocks *)
    MC_BR_TriggerPointEnable_AcpTrak_0.ProcessPoint := ADR(PP_SealProduct);
    MC_BR_TriggerPointGetInfo_AcpTrak_0.ProcessPoint := ADR(PP_SealProduct);

    (* Barrier configuration *)
    MC_BR_BarrierCommand_AcpTrak_0.ProcessPoint := ADR(PP_SealProduct);
    MC_BR_BarrierCommand_AcpTrak_0.Command := mcACPTRAK_BARRIER_ADD_TICKETS;
    MC_BR_BarrierCommand_AcpTrak_0.AdvancedParameters.TicketCount := 1;
    MC_BR_BarrierCommand_AcpTrak_0.Execute := TRUE; (* To initialize the barrier *)

    (* Counting variable *)
    CntShuttle := 1;

    (* Coupling object *)
    CoupleObjName[1] := 'CouplingAxis';
    CoupleObjName[2] := 'CouplingShuttle';

    (* Coupling position *)
    CouplingPosition[1] := 0.580;
    CouplingPosition[2] := 0.510;

```

```

END_PROGRAM

PROGRAM _CYCLIC

CASE Step OF
CHECK_TRIGGER:
    (* When the process is enabled and the assembly is ready,
    the process will be activated *)
    IF((CmdEnableProcess) AND (gTrakAsm.Status.Ready)) THEN
        MC_BR_TriggerPointEnable_AcpTrak_0.Enable := TRUE;

        (* If a shuttle passes the triggerpoint then get
        the axis reference *)
        IF((MC_BR_TriggerPointEnable_AcpTrak_0.Valid) AND
            (MC_BR_TriggerPointEnable_AcpTrak_0.EventCount > 0)) THEN
            MC_BR_TriggerPointGetInfo_AcpTrak_0.Execute := TRUE;
        END_IF
    ELSE (* Else the processpoint will be disabled *)
        MC_BR_TriggerPointEnable_AcpTrak_0.Enable := FALSE;
    END_IF

    (* If axis reference is available,
    save it and go to the next step *)
    IF EDGEPOS(MC_BR_TriggerPointGetInfo_AcpTrak_0.Done) THEN
        (* Save axis reference in local variable
        for use in step machine *)
        ShuttleAxis[CntShuttle] :=
            MC_BR_TriggerPointGetInfo_AcpTrak_0.TriggerPointInfo.Axis;

        MC_BR_TriggerPointGetInfo_AcpTrak_0.Execute := FALSE;
        Step := GET_USERDATA;
    END_IF

GET_USERDATA:
    (* The userdata is copied on a local variable *)
    MC_BR_ShCopyUserData_AcpTrak_0.Axis := ADR(ShuttleAxis[CntShuttle]);
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCopyUserData_AcpTrak_0.DataAddress := ADR(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.DataSize := SIZEOF(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.Mode := mcACPTRAK_USERDATA_GET;

    (* After copying go to the next step *)
    IF EDGEPOS(MC_BR_ShCopyUserData_AcpTrak_0.Done) THEN
        MC_BR_ShCopyUserData_AcpTrak_0.Execute := FALSE;
        Step := SEND_COUPLE_POS;
    END_IF

SEND_COUPLE_POS:
    (* Send the shuttle to the target position for coupling *)
    MC_BR_RoutedMoveAbs_AcpTrak_0.Axis := ADR(ShuttleAxis[CntShuttle]);
    MC_BR_RoutedMoveAbs_AcpTrak_0.Execute := TRUE;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Sector := ADR(Sector_SealProduct);
    MC_BR_RoutedMoveAbs_AcpTrak_0.Position := CouplingPosition[CntShuttle];
    MC_BR_RoutedMoveAbs_AcpTrak_0.Velocity := 1.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Acceleration := 20.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Deceleration := 20.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.Jerk := 0.0;
    MC_BR_RoutedMoveAbs_AcpTrak_0.BufferMode := mcABORTING;
    MC_BR_RoutedMoveAbs_AcpTrak_0.AdvancedParameters.StartDirection
        := mcDIR_UNDEFINED;
    MC_BR_RoutedMoveAbs_AcpTrak_0.AdvancedParameters.EndDirection
        := mcDIR_POSITIVE;
    MC_BR_RoutedMoveAbs_AcpTrak_0.AdvancedParameters.ShuttleOrientation
        := mcDIR_UNDEFINED;

```

```

(* When the movement is finished, go to the next step
and let in the next shuttle*)
IF EDGEPOS (MC_BR_RoutedMoveAbs_AcpTrak_0.Done) THEN
    MC_BR_RoutedMoveAbs_AcpTrak_0.Execute := FALSE;

    (* Let the second shuttle in the process station *)
    IF (CntShuttle = 1) THEN
        CntShuttle := 2;
        MC_BR_BarrierCommand_AcpTrak_0.Execute := TRUE;
        Step := CHECK_TRIGGER;
    ELSE (* Go on to the next steps *)
        CntShuttle := 1;
        Step := SET_COUPLE_OBJ;
    END_IF
END_IF

SET_COUPLE_OBJ:
    (* Place the coupling object on the slave (shuttle) *)
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Axis := ADR(ShuttleAxis[CntShuttle]);
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCouplingObjCmd_AcpTrak_0.Command := mcACPTRAK_COUPLE_OBJ_SET;
    MC_BR_ShCouplingObjCmd_AcpTrak_0.CouplingObjectName :=
        CoupleObjName[CntShuttle];

    (* When the function block is finished, go to the next step *)
    IF EDGEPOS (MC_BR_ShCouplingObjCmd_AcpTrak_0.Done) THEN
        MC_BR_ShCouplingObjCmd_AcpTrak_0.Execute := FALSE;
        Step := COUPLE_SHUTTLE;
    END_IF

COUPLE_SHUTTLE:
    (* Couple the shuttle to the axis *)
    IF (CntShuttle = 1) THEN
        MC_GearIn_0.Master := ADR(gSealAxis);
        MC_GearIn_0.Slave := ADR(ShuttleAxis[1]);
    ELSE
        MC_GearIn_0.Master := ADR(ShuttleAxis[1]);
        MC_GearIn_0.Slave := ADR(ShuttleAxis[2]);
    END_IF
    MC_GearIn_0.Execute := TRUE;
    MC_GearIn_0.Acceleration := 40;
    MC_GearIn_0.Deceleration := 40;
    MC_GearIn_0.BufferMode := mcABORTING;
    MC_GearIn_0.RatioNumerator := 1000;
    MC_GearIn_0.RatioDenominator := 1000;
    MC_GearIn_0.MasterValueSource := mcVALUE_ACTUAL;
    MC_GearIn_0.AdvancedParameters.MasterMaxVelocity := 4; (* 4m/s *)

    (* When the coupling is active, go to the next step *)
    IF EDGEPOS (MC_GearIn_0.InGear) THEN
        MC_GearIn_0.Execute := FALSE;

        (* Couple the second shuttle *)
        IF (CntShuttle = 1) THEN
            CntShuttle := 2;
            Step := SET_COUPLE_OBJ;
        ELSE (* Start the process *)
            CntShuttle := 1;
            Step := PROCESS;
        END_IF
    END_IF

PROCESS:
    (* Move the axis via mapp Cockpit *)

```

```

    (* Wait for stop command *)
    IF (cmdStopSealing) THEN
        cmdStopSealing := FALSE;
        Step := SEND_NEXT_STATION;
    END_IF

SEND_NEXT_STATION:
    (* Change the color of the Shuttle *)
    ShuttleUserData.Color := ORANGE;

    MC_BR_RoutedMoveVel_AcpTrak_0.Axis := ADR(ShuttleAxis[CntShuttle]);
    MC_BR_RoutedMoveVel_AcpTrak_0.Execute := TRUE;
    MC_BR_RoutedMoveVel_AcpTrak_0.Sector := ADR(Sector_ReleaseProduct);
    MC_BR_RoutedMoveVel_AcpTrak_0.Position := 0.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.Velocity := 1.5;
    MC_BR_RoutedMoveVel_AcpTrak_0.RouteVelocity := 1.5;
    MC_BR_RoutedMoveVel_AcpTrak_0.Acceleration := 20.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.Deceleration := 20.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.Jerk := 0.0;
    MC_BR_RoutedMoveVel_AcpTrak_0.BufferMode := mcABORTING;
    MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.StartDirection
        := mcDIR_UNDEFINED;
    MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.EndDirection
        := mcDIR_POSITIVE;
    MC_BR_RoutedMoveVel_AcpTrak_0.AdvancedParameters.ShuttleOrientation
        := mcDIR_UNDEFINED;

    (* When the movement of the shuttle is active, go to the next step *)
    IF EDGEPOS(MC_BR_RoutedMoveVel_AcpTrak_0.Active) THEN
        MC_BR_RoutedMoveVel_AcpTrak_0.Execute := FALSE;
        Step := SET_USERDATA;
    END_IF

SET_USERDATA:
    (* The modified userdata will be copied back on the shuttle *)
    MC_BR_ShCopyUserData_AcpTrak_0.Axis := ADR(ShuttleAxis[CntShuttle]);
    MC_BR_ShCopyUserData_AcpTrak_0.Execute := TRUE;
    MC_BR_ShCopyUserData_AcpTrak_0.DataAddress := ADR(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.DataSize := SIZEOF(ShuttleUserData);
    MC_BR_ShCopyUserData_AcpTrak_0.Mode := mcACPTRAK_USERDATA_SET;

    (* After copying go to the first step and wait for a new event
    and let the next shuttle into the processstation*)
    IF EDGEPOS(MC_BR_ShCopyUserData_AcpTrak_0.Done) THEN
        MC_BR_ShCopyUserData_AcpTrak_0.Execute := FALSE;

        (* Move the second shuttle to the next station *)
        IF (CntShuttle = 1) THEN
            CntShuttle := 2;
            Step := SEND_NEXT_STATION;
        ELSE (* Let the next pair of shuttles in the station *)
            CntShuttle := 1;
            MC_BR_BarrierCommand_AcpTrak_0.Execute := TRUE;
            Step := CHECK_TRIGGER;
        END_IF
    END_IF

END_CASE

(* Call all function blocks *)
MC_BR_TrgPointEnable_AcpTrak_0();
MC_BR_TrgPointGetInfo_AcpTrak_0();
MC_BR_ShCopyUserData_AcpTrak_0();
MC_BR_RoutedMoveAbs_AcpTrak_0();
MC_BR_ShCouplingObjCmd_AcpTrak_0();

```

```
MC_GearIn_0();
MC_BR_RoutedMoveVel_AcpTrak_0();

MC_BR_BarrierCommand_AcpTrak_0();
(* When adding tickets is done, reset the execute *)
IF EDGEPOS(MC_BR_BarrierCommand_AcpTrak_0.Done) THEN
    MC_BR_BarrierCommand_AcpTrak_0.Execute := FALSE;
END_IF

END_PROGRAM
```


Automation Academy

Your knowledge advantage

The Automation Academy provides **targeted training** courses for our customers as well as for our own employees. Expand your skills in the field of automation technology and learn to independently implement **efficient automation solutions** using B&R systems.

Decide for yourself which **learning concept** you prefer!



Classroom Learning



Virtual Classroom
Learning



Online courses

Classroom learning

B&R offers **standard seminars** at all B&R locations. Services include seminar documents, effective communication of training course content by experienced trainers and an Automation Diploma. A combination of group work and self-study provides the high level of flexibility needed to maximize the learning experience.

Virtual classroom

Remote Lectures supplement B&R's continuing education portfolio with a virtual classroom, offering an alternative to our on-site seminars. Selected content from our standard seminars is offered online. In addition to remote learning methods, powerful simulation tools and secure remote maintenance are used.

Online courses

Take control of the content and learn at your own pace. With B&R **online courses**, you can take your first steps in the world of B&R automation at any time. Based on a comprehensive narrative, you will independently work out how to use our products. The mix of different media allows a logical sequence to be followed when learning as well as a targeted choice of information to be used as a reference.

Contact

Would you like additional training? Are you interested in finding out what the B&R Automation Academy has to offer? If so, this is the right place.

Access additional information here:

<https://www.br-automation.com/de/academy/>

Enjoy your next training course!



AutomationAcademy





B&R
Industrial Automation GmbH
A member of the ABB Group
B&R Straße 1
5142 Eggelsberg, Austria
office@br-automation.com

t +43 7748 6586-0
f +43 7748 6586-26

br-automation.com

